

Database Design Using E-R Model — Chapter 6

2025 年 10
月

School of Computer Science



北京邮电大学

Outline

1. DB Design Process
2. E-R Model (conceptual design)
3. Reducing E-R diagrams to relational schemas (logical design)

6.1 Overview of Design Process

- **Database design** consists of two **phases**
 - **Analyzing** of requirements
 - **what data** should be **stored**
 - **what operations/transaction** are **conducted** such as **insert, delete, update**
 - **Designing** of DB schemas with three-level **abstract**
 - **conceptual design**
 - **logical design**
 - **physical design**

Application problems in real worlds



Requirements analysis



Specification of user requirements



Conceptual DB design



DB conceptual schema , i.e. E-R diagram



Logical DB design



DB logical schema , i.e. relational data schema



Physical DB design



DB physical schema, e.g. physical storage structure

**DB
Design
Phases**

Overview of the Design Process (cont.)

■ 数据库应用系统 (DBAS) 设计

▣ 1. 生命周期包括五个阶段：

项目规划、需求分析、系统设计、实现与部署、运行管理与维护等

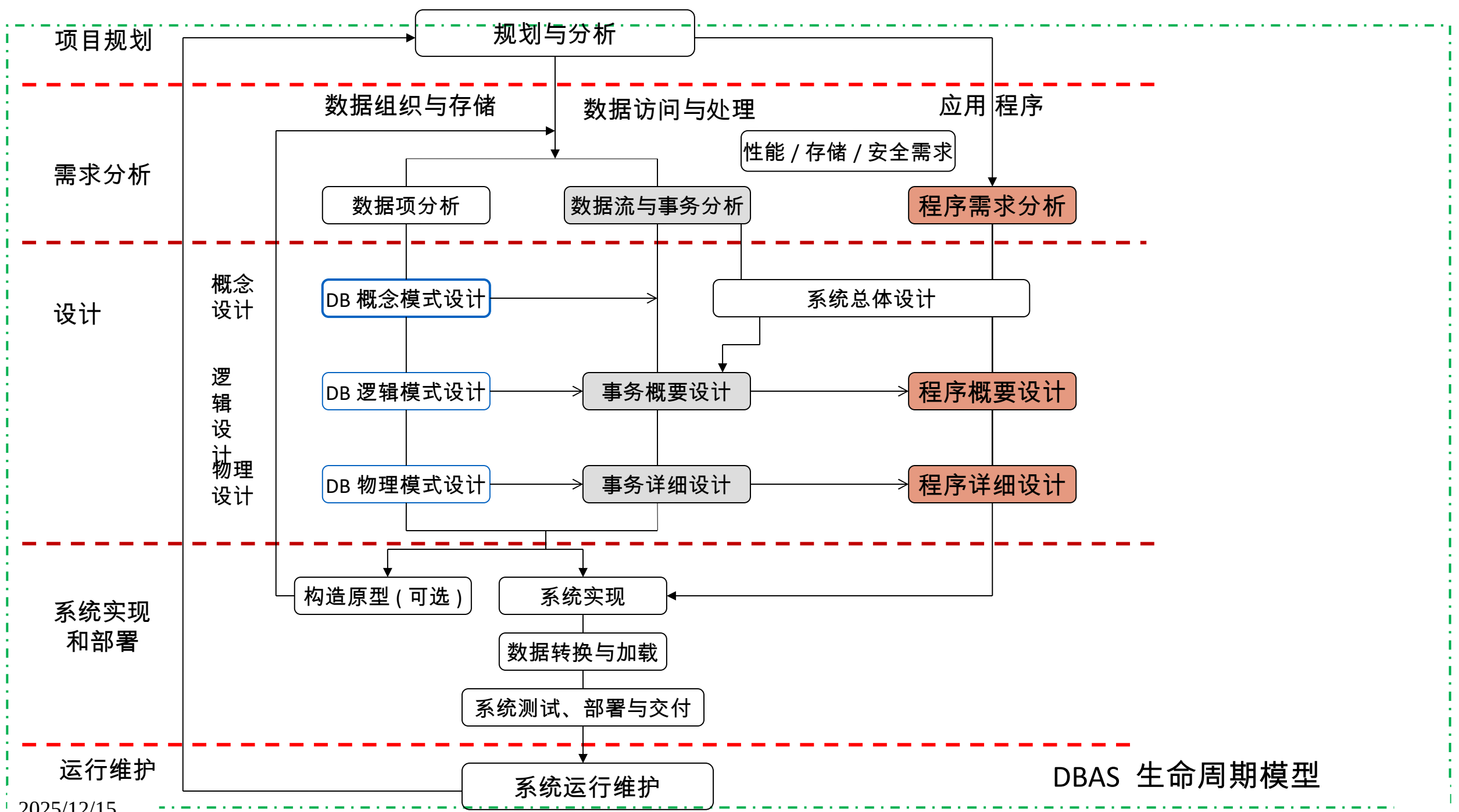
▣ 2. 根据软件组成和各自功能，分为三条设计主线：

数据组织与存储设计、**数据访问与处理设计**、**应用设计**

分别用于设计数据库、数据库事务和应用程序

▣ 3. 根据数据库系统三级模式结构，设计阶段分为三个步骤：

概念设计、逻辑设计、物理设计，



6.2 The Entity-Relationship Model

- **Def:** ER model **facilitates** database design of DB
 - represents the overall logical structure
 - mapping the meanings and interactions of **real-world enterprises** onto a **conceptual schema**
- ER model employs three concepts
 - **entity sets**
 - **relationship sets**
 - **attributes**
- ER model has a **diagrammatic representation**, the ER diagram

6.2.1 Entity Set (实体集)

- **Def:** an **entity** is an object that exists **distinguishable** from other objects.
 - Example: a **specific** person, company
- **Def:** an **entity set** is a set of entities of the same type
 - sharing the same properties.
 - Example: sets of persons, companies
- **Note:** An entity is represented by a set of **attributes**;
 - descriptive properties of an entity set
 - Example: *instructor = (ID, name, street, city, salary)*
course = (course_id, title, credits)
- **Def:** A subset of the attributes form a **primary key**,
 - uniquely identifying each member of the entity set.

Entity Sets (cont.)

- **Note:** A entity consists of **all values of all attributes**

- i.e., Entity set

instructor

$\{(\text{instructor_ID}, \text{instructor_name}) :$

$\text{instructor_ID} \in D_1,$

$\text{instructor_name} \in D_2\}$

$$\subseteq D_1 \times D_2$$

Instructor_ID instructor_name

76766	Crick
45565	Katz
10101	Srinivasan
98345	Kim
76543	Singh
22222	Einstein

instructor

student_ID student_name

98988	Tanaka
12345	Shankar
00128	Zhang
76543	Brown
76653	Aoi
23121	Chavez
44553	Peltier

student

6.2.2 Relationship Sets

- **Def:** A **relationship** is an **association** among several entities

Example: 44553 (Peltier) advisor 22222 (Einstein)
 student entity **relationship set** *instructor* entity

- **Def:** A **relationship set** is a **mathematical relation** among **n** entities ($n \geq 2$), taken from entity set

$$\{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, e_2 \in E_2, \dots, e_n \in E_n\}$$

where $(e_1, e_2, \dots, e_n)$ is a relationship

- Example: $(44553, 22222) \in \text{advisor}$

- ❑ **Note:** more than one relationship set among the same entity sets

Relationship Sets (Cont.)

- **Note:** The entity sets $E_1, E_2, \dots, E_n$ **participate in** relationship set R
- **Def:** **A relationship instance** represents **an association** between the named entities in the real-world enterprise
- e.g. **a relationship instance** between *Crick* and *Tanaka*

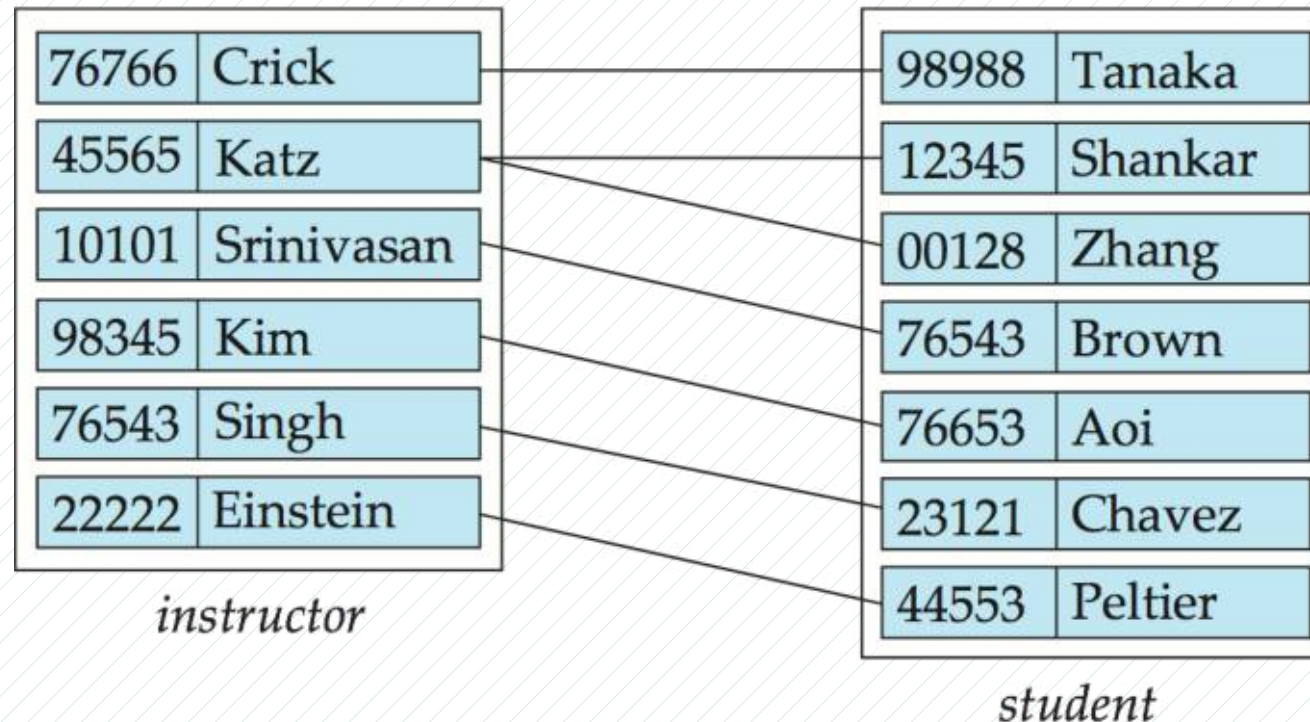
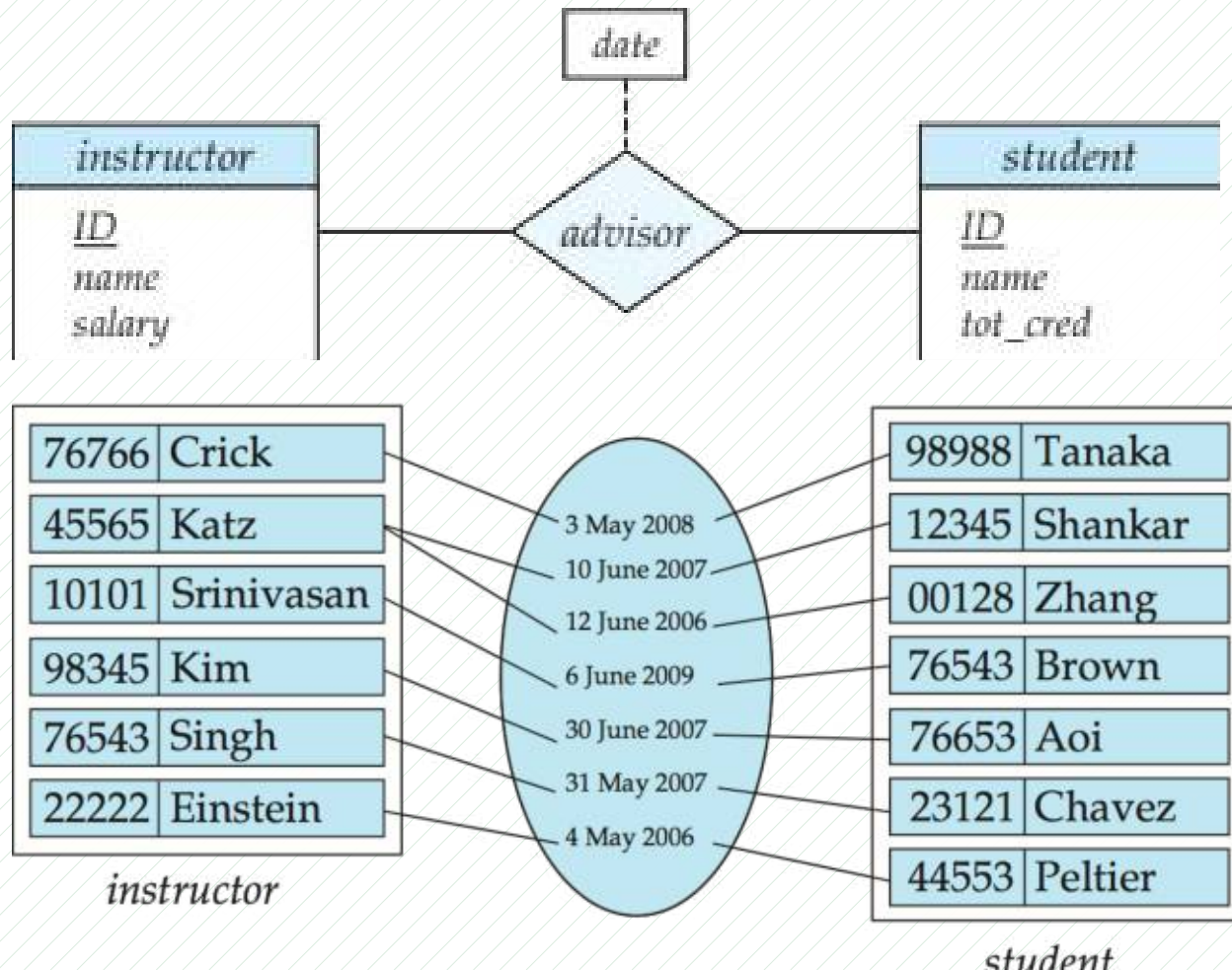


Fig. 6.2 Relationship Set **advisor**

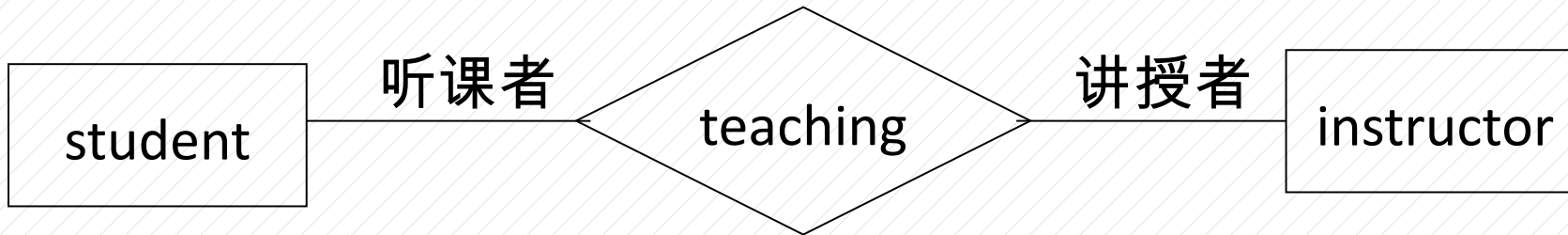
Attributes with Relationship Sets

- **Def:** An **attribute** can be associated with a **relationship set**.
- **Example:** The **relationship** set *advisor* between *instructor* and *student* has an attribute **date** when *student* associated with *instructor*



Role in Relationship Sets

- **Note** : Entity sets of a relationship **neednot** be distinct
 - each occurrence of an **entity set** plays a **role** in relationship
- **Role**
Def : a role of entity is a **function** that an entity plays in a relationship
i.e. : The **labels** 听课者 and 讲授者 are called **roles**.



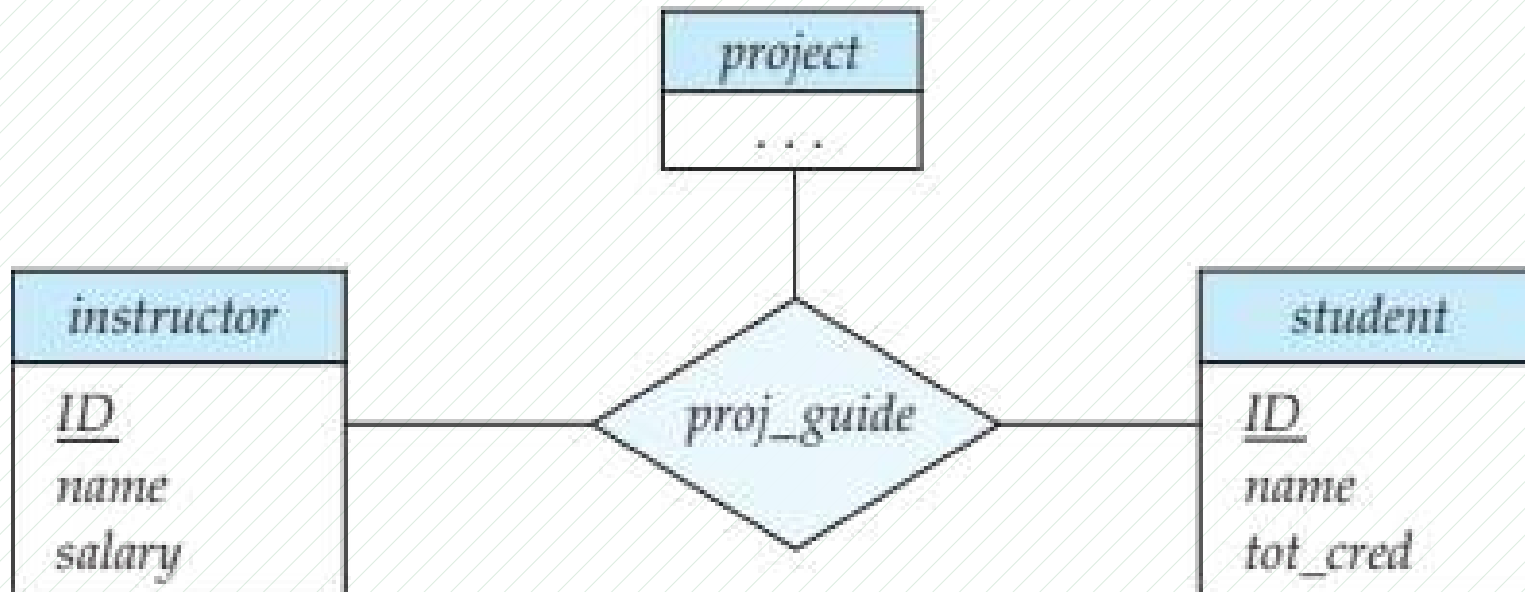
Degree of a Relationship Set vs Non-binary Relationship Sets

■ Binary relationship

- involve **two** entity sets (degree two).
- most relationship sets are binary.

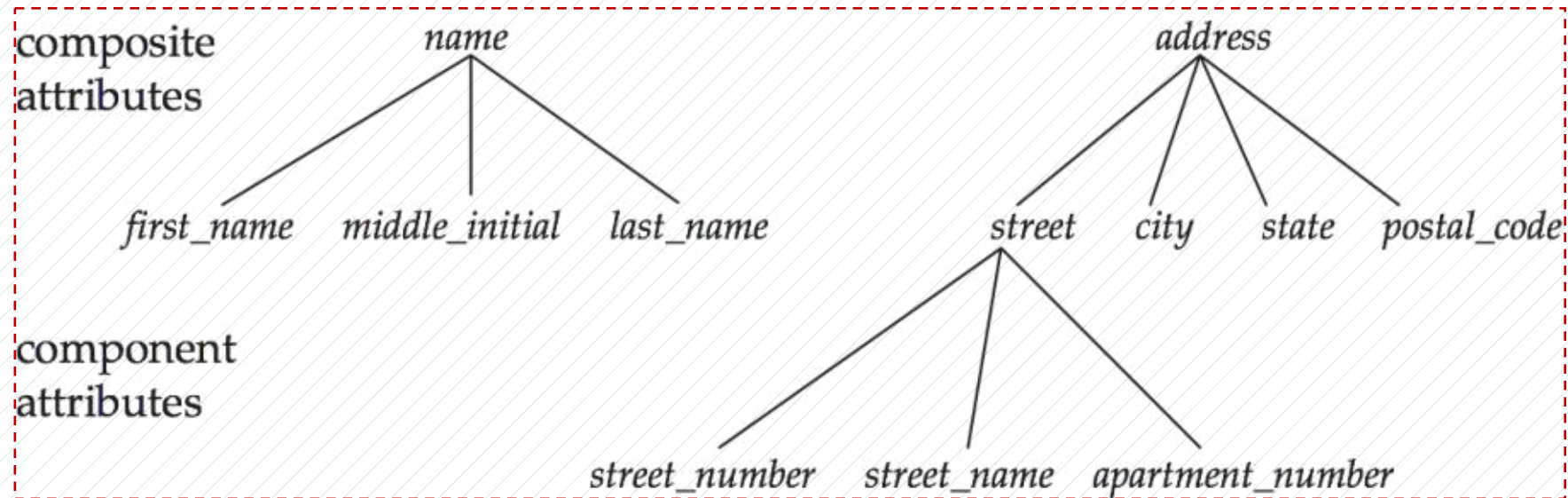
■ Ternary Relationship

- Example: *students* work on research *projects* under guidance of an *instructor*
 - relationship *proj_guide* is **ternary** among *instructor*, *student*, and *project*



6.3 Complex Attributes

- **Def:** The **domain** or value set of the attribute
 - The set of **permitted values**
- **Attribute types:**
 - **simple** and **composite** attributes
 - **single-valued** and **multivalued** attributes, e.g. phone_numbers



- **derived attributes**
 - can be computed from other attributes, e.g. age, given date_of_birth

Null in Attributes

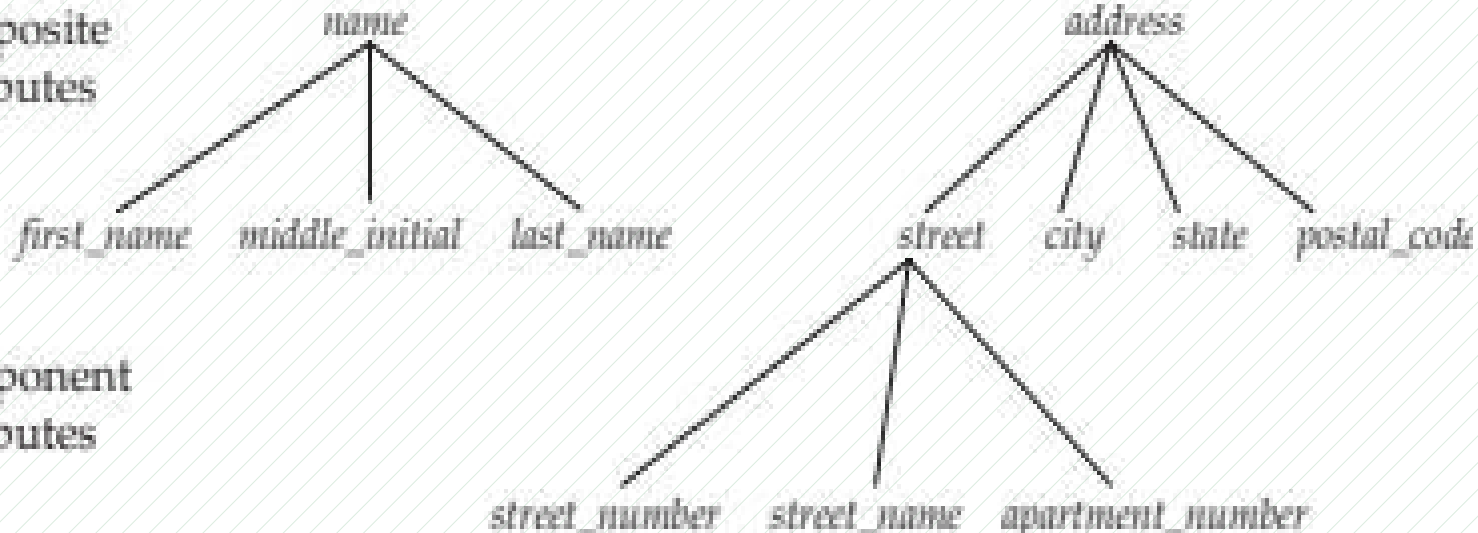
- **Null** value for a attribute means
 - ▣ the attribute **not applicable** for the entity, **not existing**
 - ▣ the **value** for the attribute **exists**, but is “unknown”

Composite Attributes

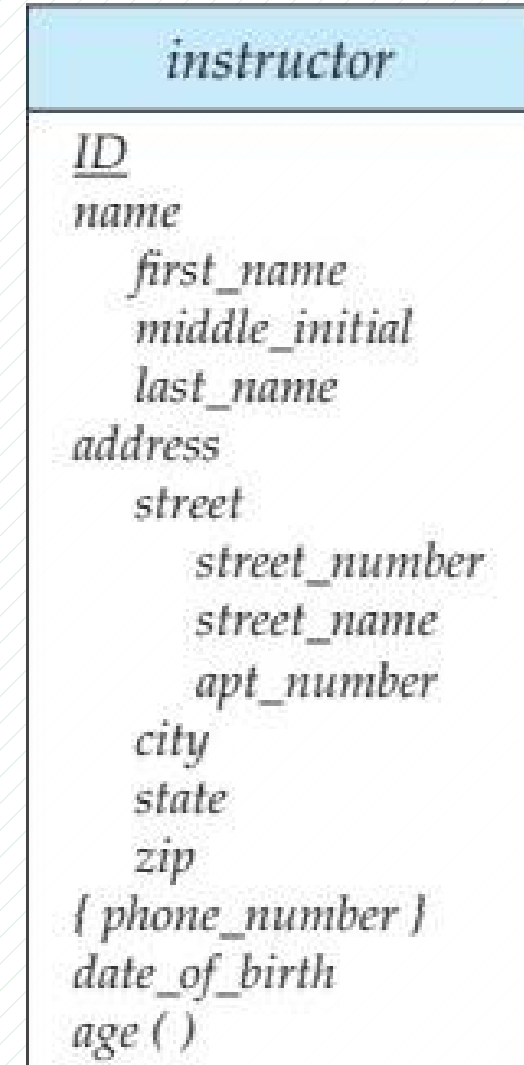
- **Composite attributes** allow us to divide attributes into subparts

composite
attributes

component
attributes



- **Representing Complex Attributes in ER Diagram**



Redundant Attributes

- **Suppose** we have entity sets:
 - *instructor*, with attributes: *ID*, *name*, *dept_name*, *salary*
 - *department*, with attributes: *dept_name*, *building*, *budget*
- **Model** : each **instructor** has an **department** using relationship set *inst_dept*
- The attribute *dept_name* appears in both entity sets.

Since it is **the primary key** for the entity set *department*,
it replicates information present in the *relationship*.
It is therefore **redundant** in the entity set *instructor*.

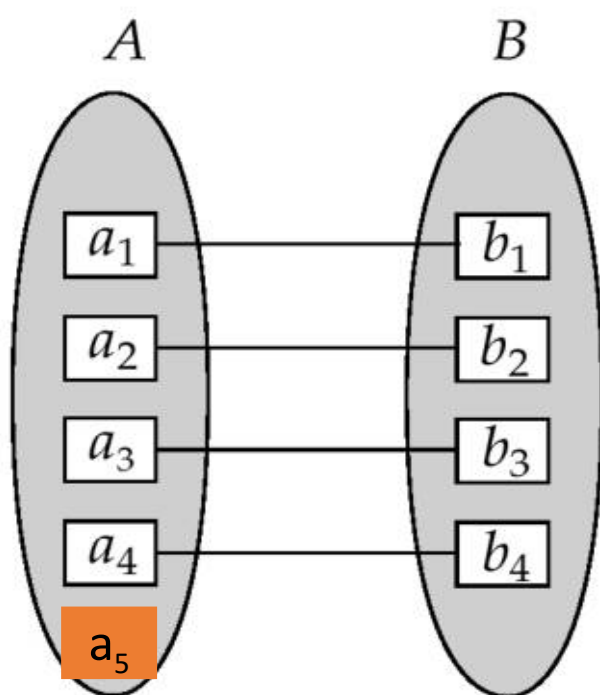
6.4 Mapping Cardinalities Constraints

- Semantic/constraints
 - as **integrity constraints**, to maintain **consistency**
- Integrity constraints, on *E* or *R*
 - **mapping cardinalities**
 - **participation constraints**
 - **cardinality limits**
 - **keys**

Mapping Cardinalities

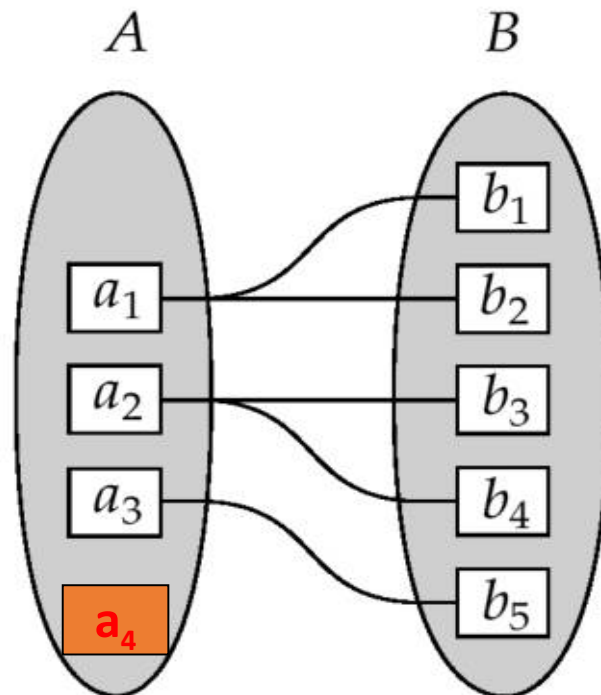
- **Mapping cardinalities**
 - **The number of entities** to which another entity can be associated via a **relationship set**
- **Def:** For a binary relationship set **R** , from A to B, the mapping cardinality must be:
 - one to one
 - one to many
 - many to one
 - many to many

—注意 many-to-one 和 one-to-many 定义和方向 !



(a)

One to one



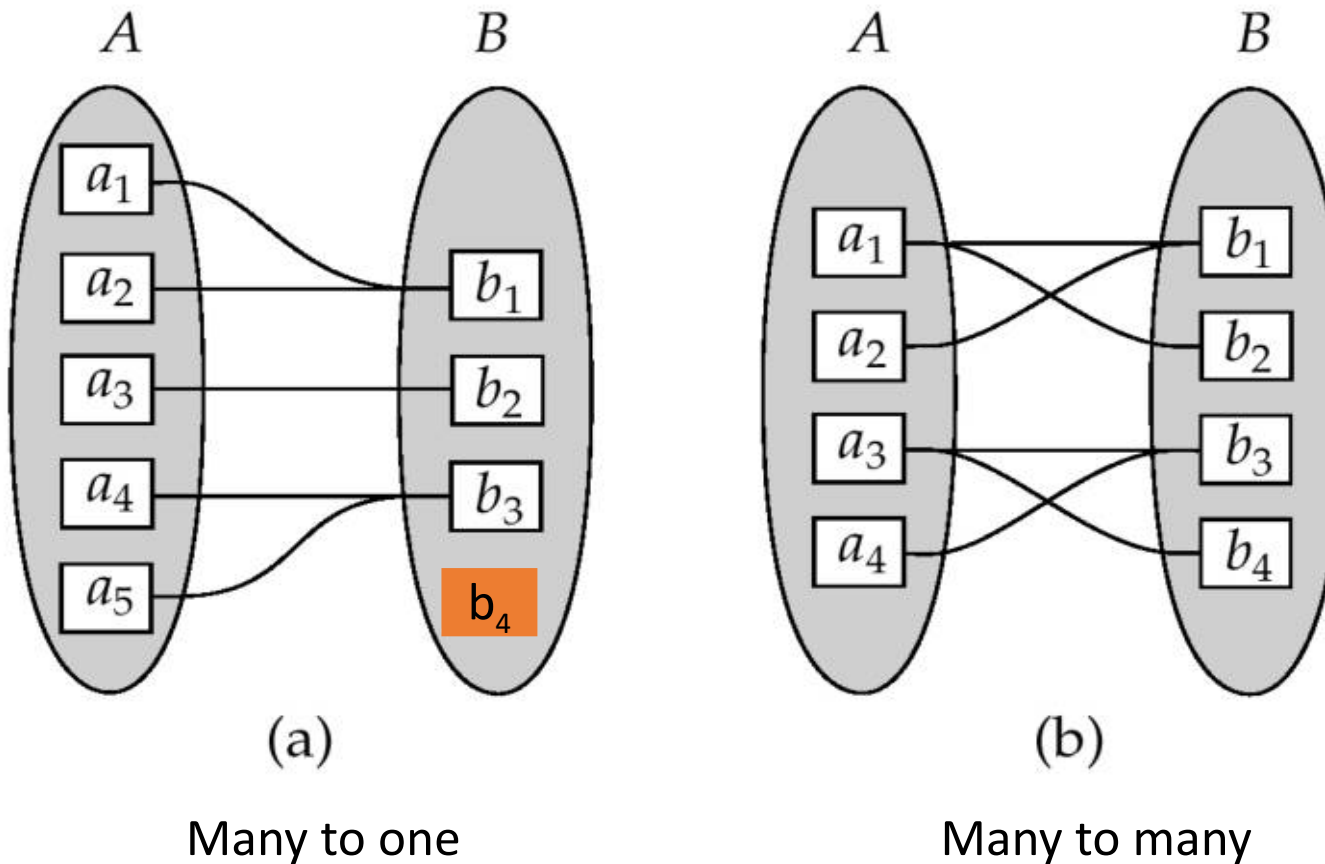
(b)

One to many

Note: some elements in A or B may **not be mapped** to any elements in the other set

Fig. 6.5 Mapping cardinalities

— 注意 many-to-one 和 one-to-many 定义和方向！

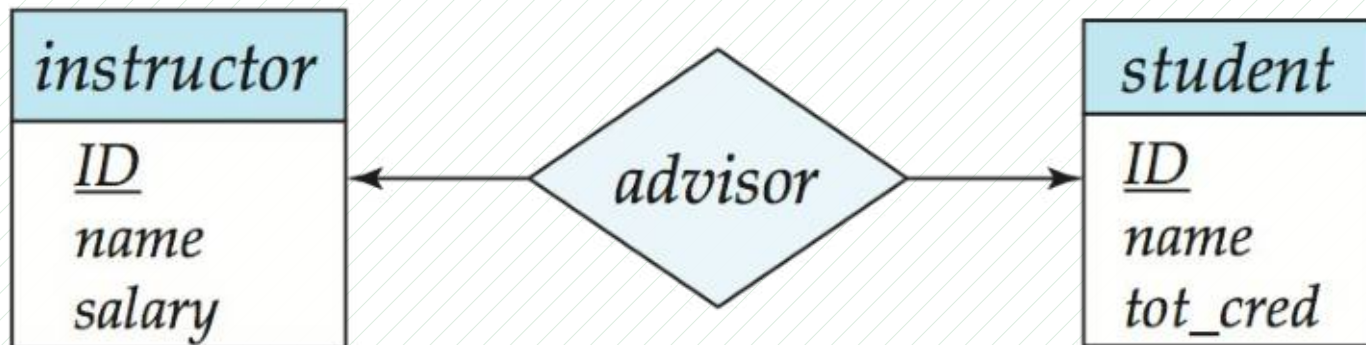


Note: Some elements in **A** or **B** may not be mapped to any elements in the other set

Fig. 6.6 Mapping cardinalities

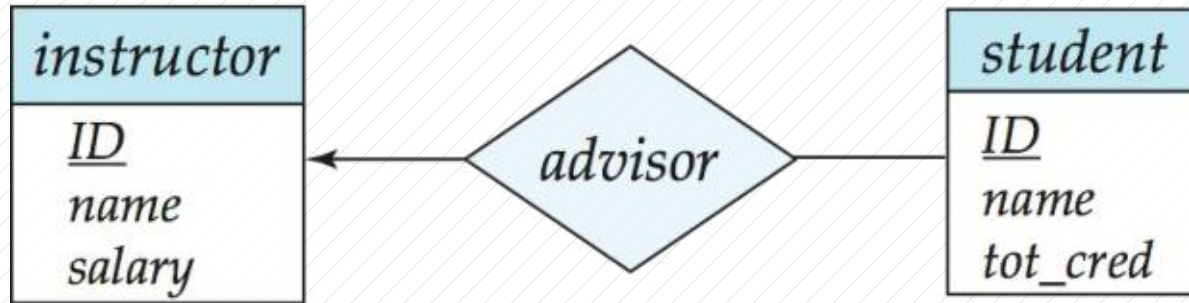
Cardinality Constraints in ER Diagram

- **Note** : We express **cardinality constraints** by drawing either a directed line ($\rightarrow$), signifying “one”, or an undirected line ($-$), signifying “many” between the **relationship** set and the **entity** set.
- i.e., **One-to-one** relationship between an *instructor* and a *student* :
 - ▣ A student is associated with **at most** one *instructor* via the relationship *advisor*



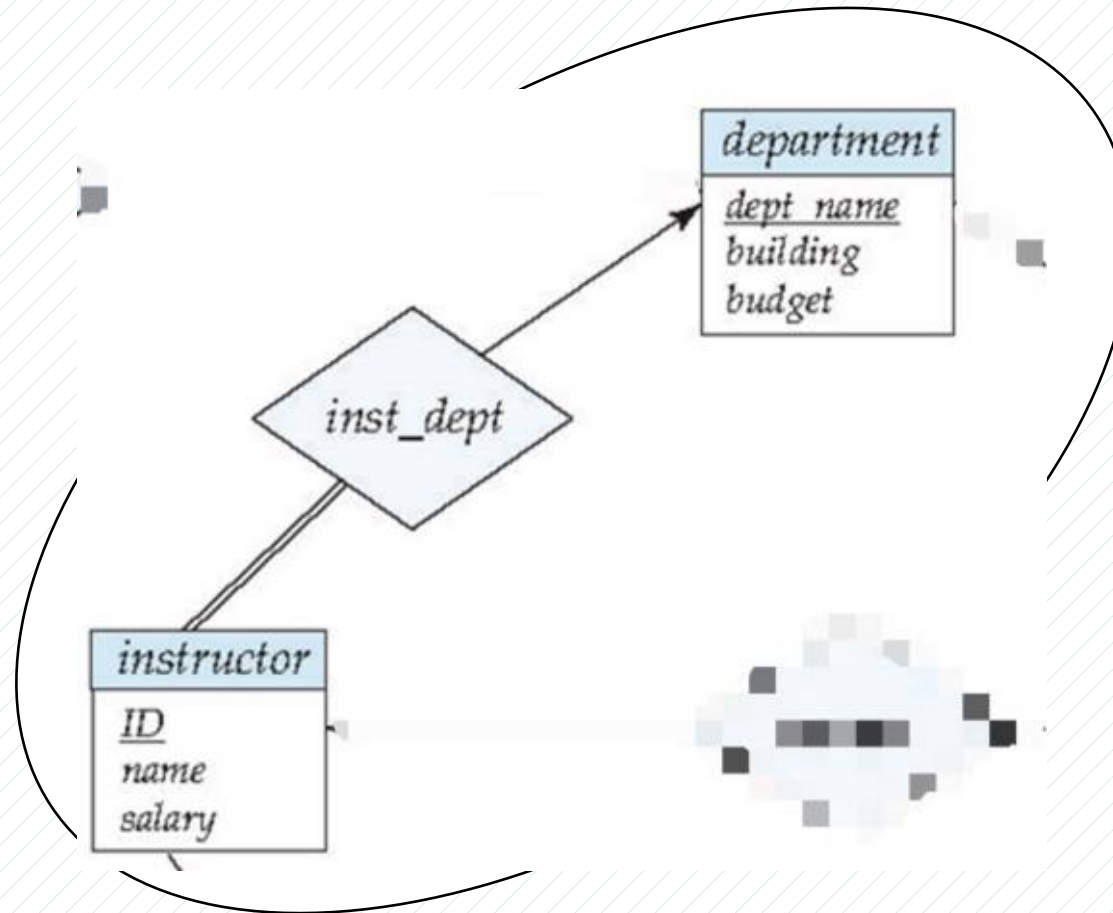
Mapping Cardinalities

- i.e., **one-to-many** relationship between an *instructor* and a *student*
 - **an** instructor is associated with **several** (including **0**) students via *advisor*
 - **a** student is associated with **at most one** instructor via *advisor*,



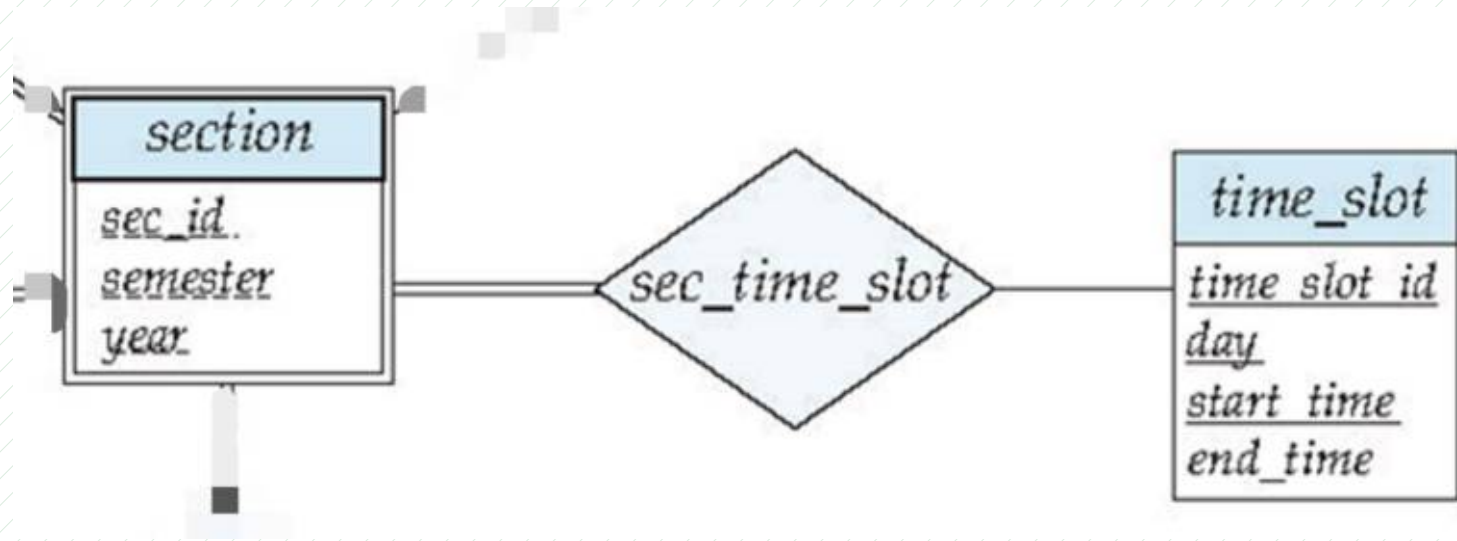
Mapping Cardinalities

- **many-to-one** relationship between an *instructor* and a *department*
 - ▣ an instructor is associated with at most one department via *inst_dept*,
 - ▣ a department is associated with several (at least one) instructor via *inst_dept*



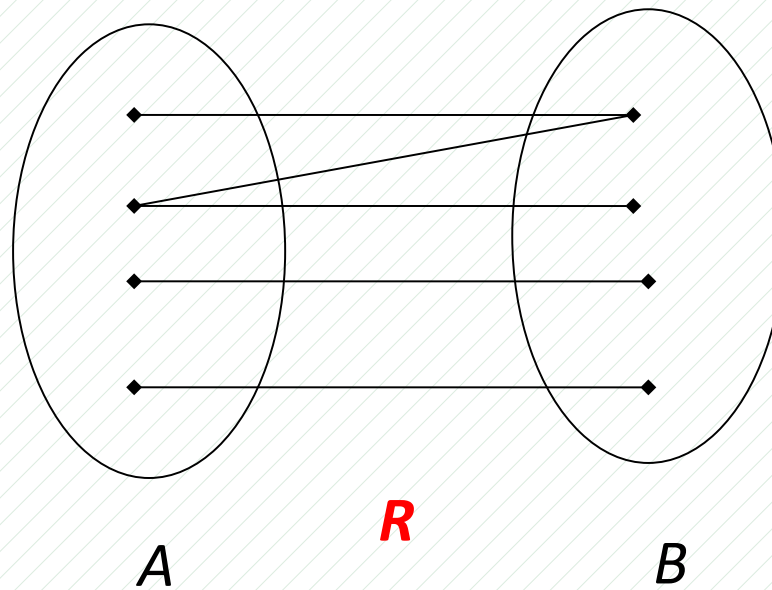
Mapping Cardinalities

- i.e., many-to-many relationship between *section* and a *timeslot*
 - ▣ A *section* is associated with several (at least one) *timeslots* via *sec_time_slot*
 - ▣ A *timeslot* is associated with several (possibly 0) *sections* via *sec_time_slot*



Participate Constraints

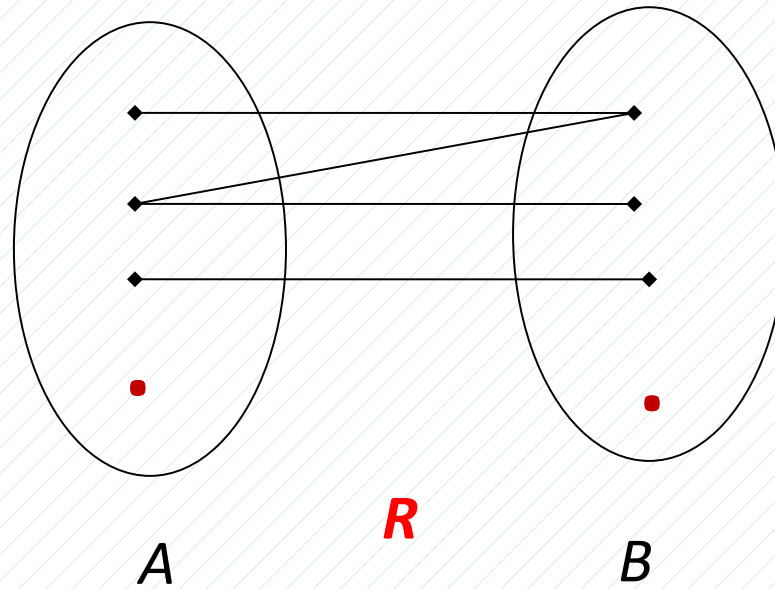
- **Def:** The participation of an **entity** *E* in a relationship *R* is **total**, if
 - every entity in *E* participates in **at least one** relationship in *R*
 - e.g. participation of *instructor* in *inst_dept* is total
 - every *instructor* **must** have a *department* associated to it via *inst_dept*



(b) total participation

Participate Constraints

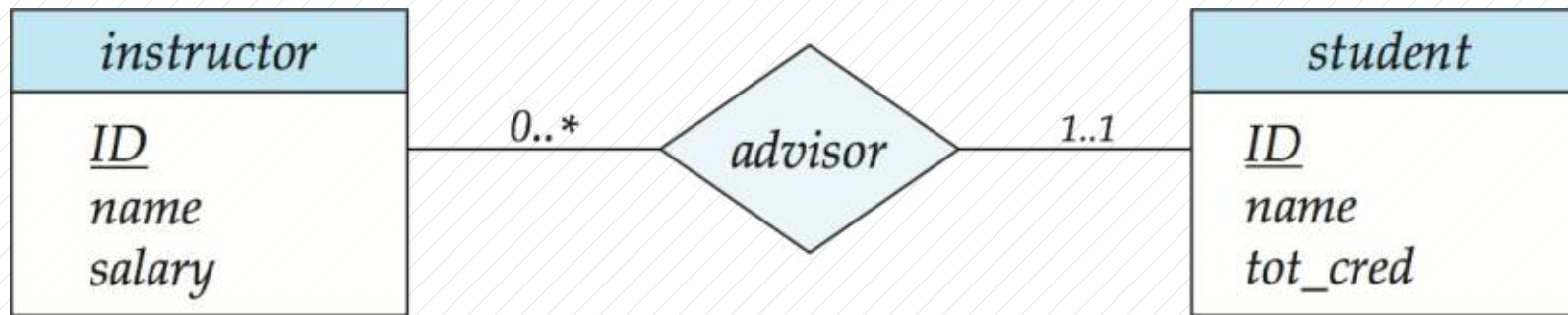
- **Def:** The participation of an entity ***E*** in a relationship ***R*** is **partial**, if
 - some entities in ***E*** may not participate in any relationship in ***R***
 - e.g. participation of *student* in *advisor* is partial, because some students maybe have no advisors



(a) partial participation

Cardinality Limits for Participation

- **Def:** Cardinality limits (基数界限) express **quantitative** constraints on participation
- e.g. *instructor*, *student*, *advisor*
 - 每个 *instructor* **最多** 可以指导多个 *student*, **最少** 可以指导 0 个 *student*
 - 每个 *student* **最少** 有 1 个指导 *instructor* , **最多** 也只有 1 个指导 *instructor*

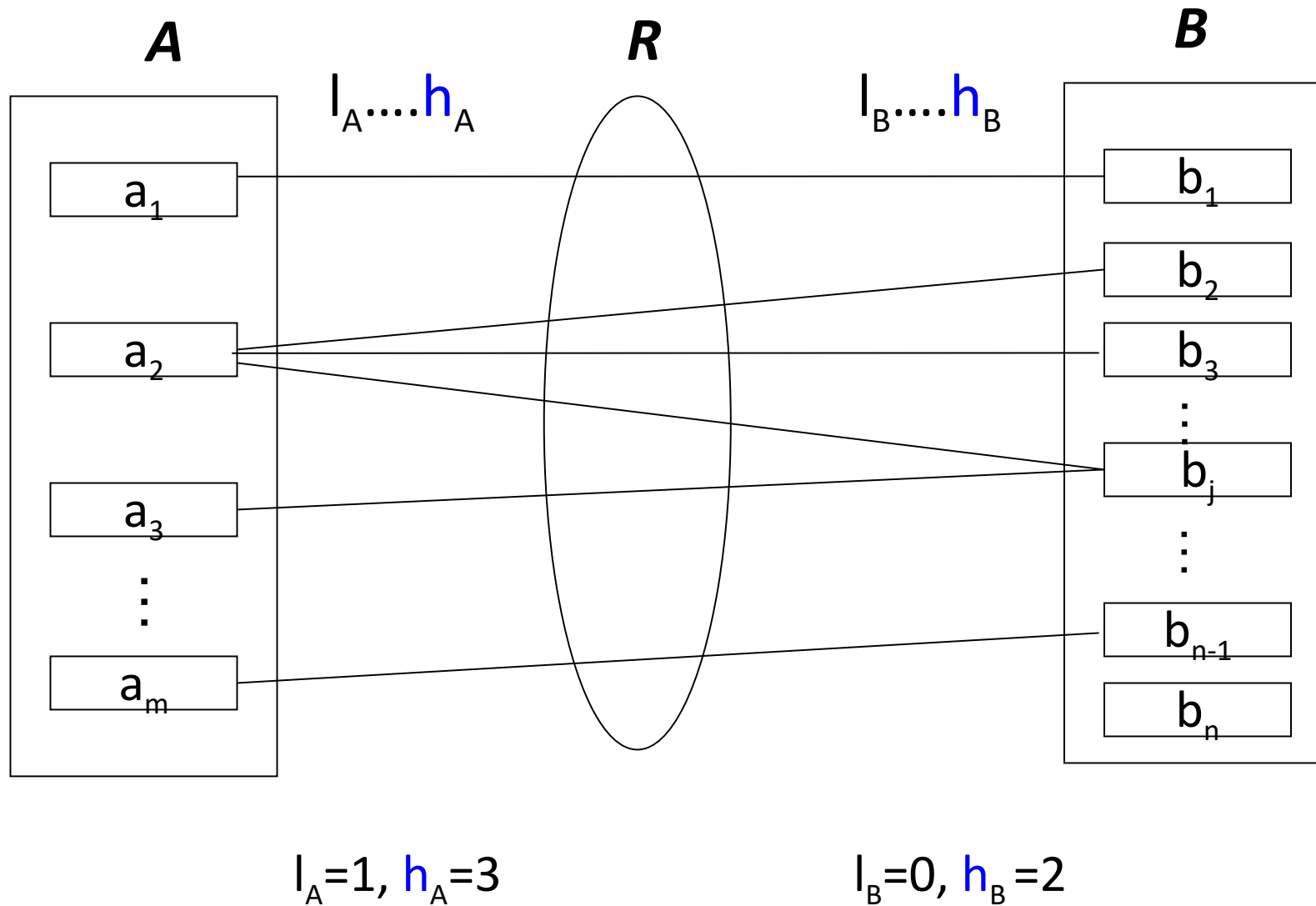


A instructor can advise 0 or more students.

A student must have 1 advisor; whereas cannot have multiple advisors

Cardinality Limits for Participation

- 联系 R 关联了 entity sets A 和 B, 为定量描述 A 参与**联系** R 的 entity A 与 entity B 的 **mapping cardinality**, 引入**实体参与联系**的 **cardinality limits**
- **Def:** A **参与** R 的**基数**下界 l_A 和上界 h_A
 - ▣ A 中的每个实体 a 通过 R 关联了最少 l_A 个、最多 h_A 个 B 中实体 b
 - l_A : 对 A 中的每个实体 a, B 中至少有 l_A 个实体 b 通过 R 与其对应 / 关联
 - h_A : 对 A 中的每个实体 a, B 中至多有 h_A 个实体 b 通过 R 与其对应 / 关联



cardinality limits

Cardinality Limits for Participation

- Note
 - ▣ A maximum value * indicates **no limit**
 - ▣ *in some textbooks*, $\langle l_A, h_A \rangle$ is put at the side of the entity B
- Cardinality limits vs **total/partial participation**
 - ▣ minimum value $l_A = 0$: **A** is **partial** participation of **R**
 - ▣ minimum value $l_A > 0$: **A** is **total** participation of **R**,
equivalent to **double line**

Cardinality limits vs Mapping cardinality

- **Def:** 联系 R 关联了实体集 A 和 B , 利用**基数界限** $\langle l_A, h_A \rangle$ 的 h_A 、**基数界限** $\langle l_B, h_B \rangle$ 的 h_B ,

得到联系 R 的映射基约束 $\langle h_B, h_A \rangle$

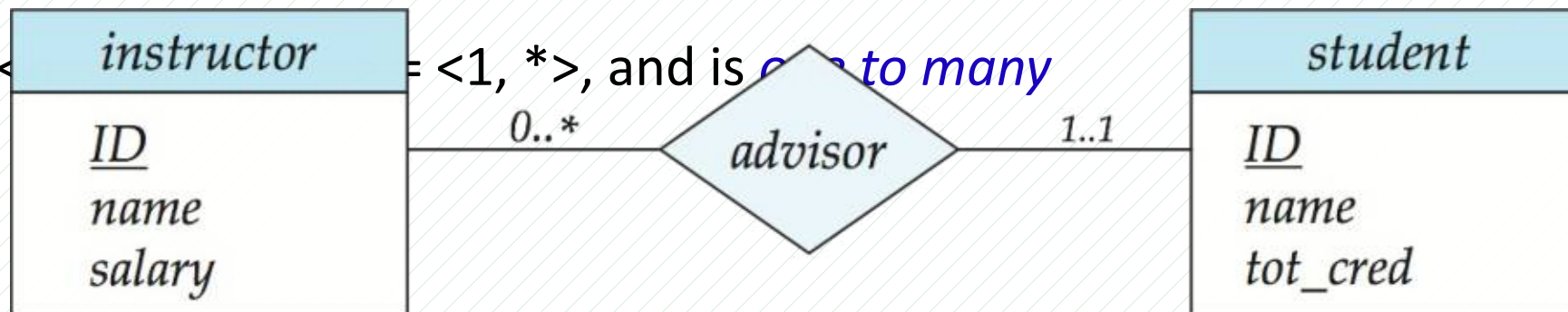
□ $\langle h_B, h_A \rangle$ 表示了联系 R 的从 A 到 B 的映射基约束 !!

- e.g. considering mapping cardinality from *instructor* to *student*

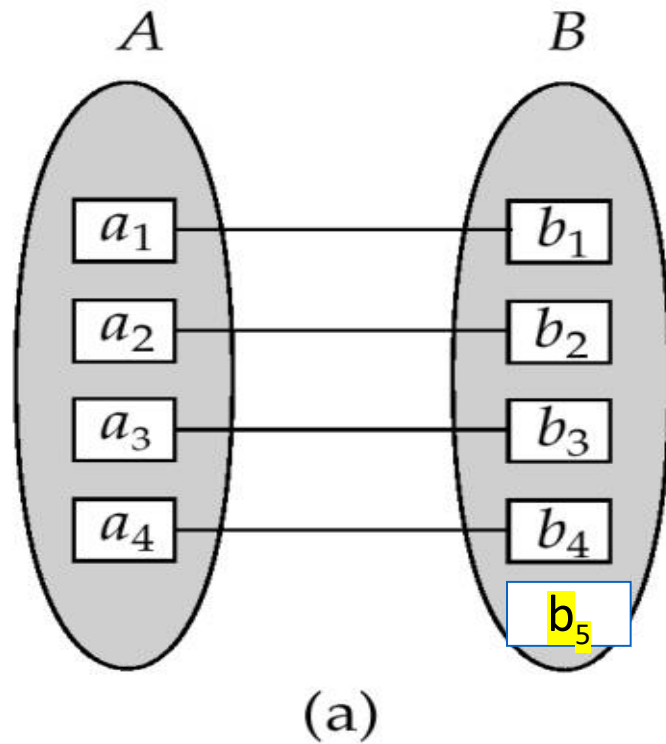
□ mapping cardinality from *instructor* to *student* depends on $\langle h_{\text{student}}, h_{\text{instructor}} \rangle$

□ *student*, $h_{\text{student}} = 1$, *instructor*, $h_{\text{instructor}} = *$

□ $\langle h_{\text{student}}, h_{\text{instructor}} \rangle = \langle 1, * \rangle$, and is *one to many*



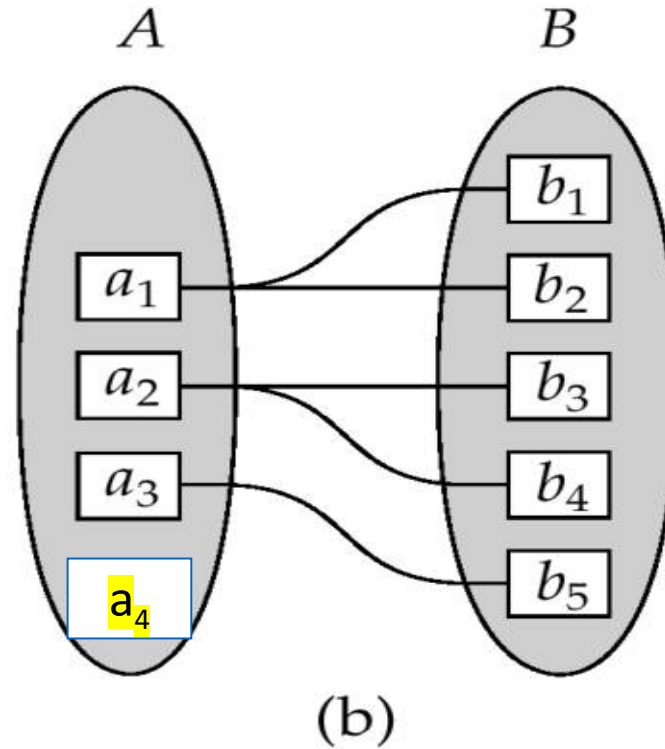
Associations between cardinality limits and mapping cardinality



From A to B, **one** to **one** (1:1),

$$\langle l_A, h_A \rangle = \langle 1, 1 \rangle,$$

$$\langle l_B, h_B \rangle = \langle 0, 1 \rangle;$$

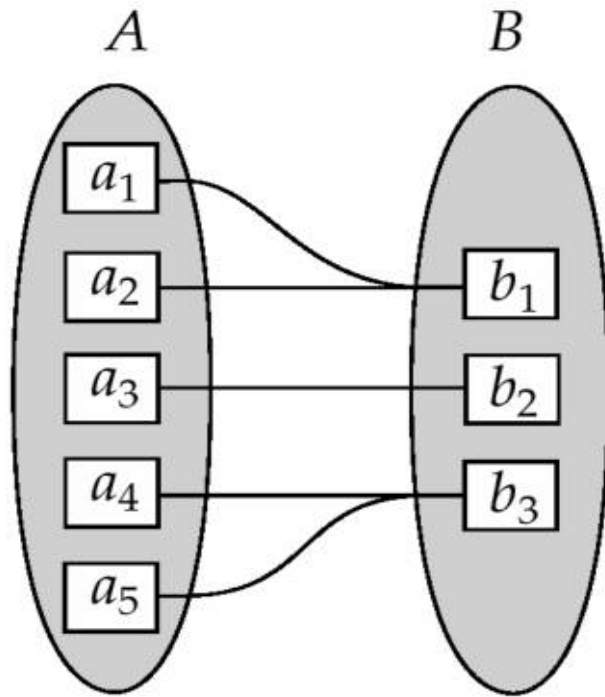


From A to B, **one** to **many** (1:2),

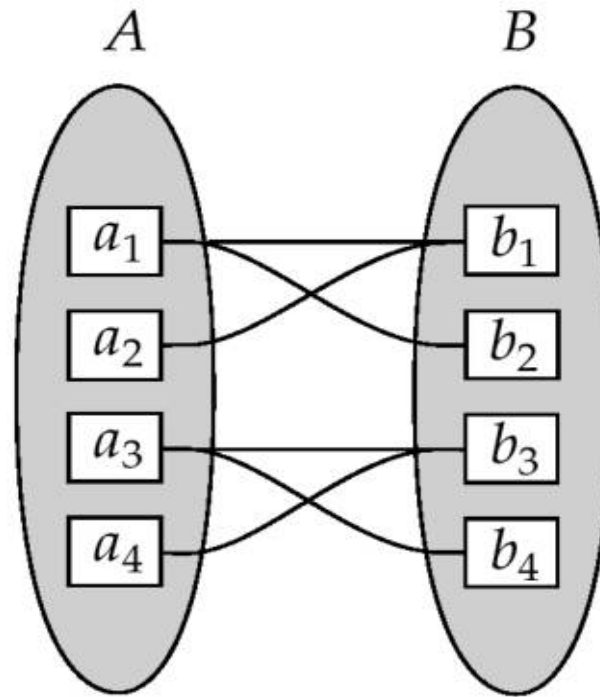
$$\langle l_A, h_A \rangle = \langle 0, 2 \rangle,$$

$$\langle l_B, h_B \rangle = \langle 1, 1 \rangle$$

Associations between cardinality limits and mapping cardinality



(a)



(b)

Form A to B, many to one (2:1),

$$\langle l_A, h_A \rangle = \langle 1, 1 \rangle,$$

$$\langle l_B, h_B \rangle = \langle 1, 2 \rangle$$

Form A to B, many to many (2:2)

$$\langle l_A, h_A \rangle = \langle 1, 2 \rangle$$

$$\langle l_B, h_B \rangle = \langle 1, 2 \rangle$$

6.5 Primary Keys

- **Def: Key** is a set of attributes (a **entity set** or **relationship set**),
 - the values of these attributes in one **entity** to **uniquely** distinguish this entity from others
 - the values of these attributes in one **relationship** to **uniquely** identify the relationship
- **Note** : Keys include
 - superkey (超键), candidate key (候选键), primary key (主键)
- **Note** : keys for
 - Entity sets
 - Relationship sets
 - Weak entity sets

Keys For Entity Set

- **Def:** A **super key** of an entity set is a set of **one or more** attributes, whose values **uniquely** determine each entity in the entity set
 - e.g. {instructor_id , instructor_name}
 - the super key may contain **extraneous** attributes
 - e.g instructor_name
- **Def:** A candidate key is the **minimal** super key
 - **non-redundant** super key
 - e.g. instructor-id is the candidate key of instructor

Keys For Entity Sets (cont.)

- **Def:** The **primary key** is a candidate key chosen as the **principal** means of identifying entities within an entity set
 - ▣ although **several** candidate keys may exist, **one** of the candidate keys is selected to be the **primary key**

Keys For Relationship Sets

- **Def:** Keys for relationship sets R on entities $E_1, E_2, \dots, E_n$
 - ▣ $R = \{(e_1, e_2, \dots, e_n) \mid e_1 \in E_1, \dots, e_n \in E_n\}$
 $\subseteq E_1 \times E_2 \times \dots \times E_n$
 - ▣ Uniquely distinguish each relationship **instances** $\{(e_1, e_2, \dots, e_n) ?$
 - ▣ R is the combination of $E_1, E_2, \dots, E_n$,
each E_i can be uniquely distinguished by **primary**_key(E_i), $1 \leq i \leq n$,
so **all** attributes in **primary**_key(E_1), **primary**_key(E_2), ..., **primary**_key(E_n)
can be used to recognize $(e_1, e_2, \dots, e_n)$

Keys For Relationship Sets (cont.)

- **Def:** The **super**_key for R
 - $\text{primary_key}(E_1) \cup \text{primary_key}(E_2) \dots \cup \text{primary_key}(E_n)$
 - e.g. (InstructorID, StudentID) is the super key of **advisor**
 - **Note:**
 - if the attribute names of primary-keys are **not unique**, the **attributes with the same names** should be **renamed**
- **Def:** The candidate keys for R
 - **minimal, non-redundant** super keys

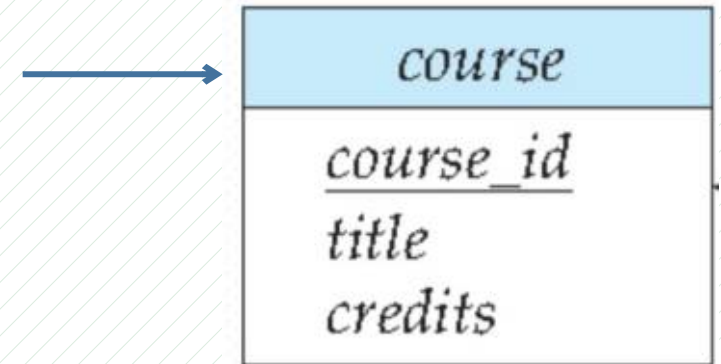


Choice of Primary key for Binary Relationship

- The **candidate** keys or **primary** key for ***R*** among entity sets ***A*** and ***B*** can be decided with the mapping cardinality
 - ▣ ***R*** is *many-to-many*,
$$\text{primary_key}(R) = \text{primary_key}(A) \cup \text{primary_key}(B)$$
 - ▣ ***R*** is *many-to-one from A to B*
$$\text{primary_key}(R) = \text{primary_key}(A)$$
 - ▣ ***R*** is *one-to-many*
$$\text{primary_key}(R) = \text{primary_key}(B)$$
 - ▣ ***R*** is *one-to-one*,
$$\text{primary_key}(R) = \text{primary_key}(A) \text{ or } \text{primary_key}(B)$$

6.5.3 Weak Entity Sets

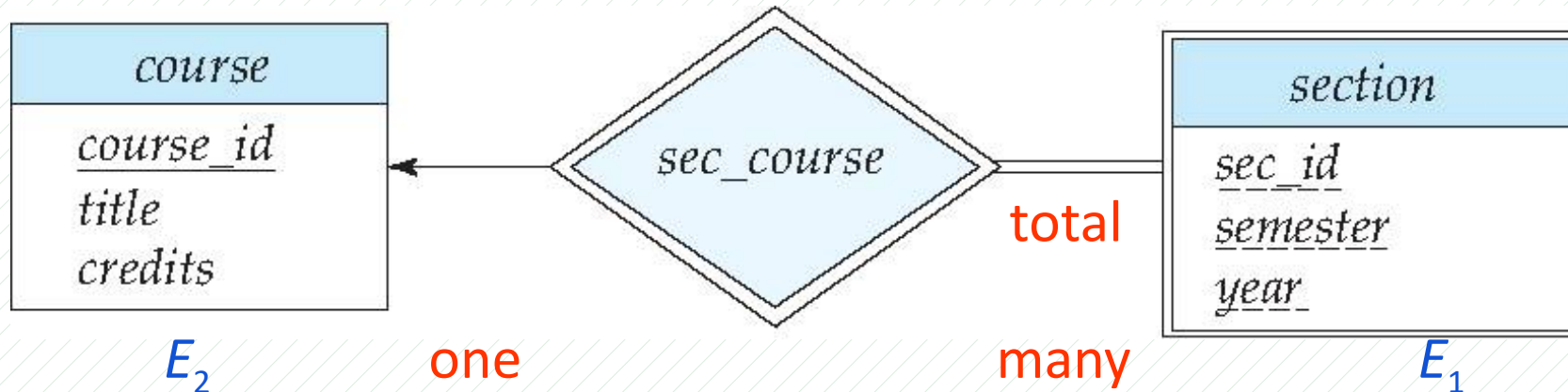
- **Def:** An entity set that does **not** have a primary key is a **weak entity set**
- Consider **section** entity set (*course_id*, semester, year, sec_id) , entities in **section** are related to entities in **course**.
- **方案 1** : 引入联系 , 带来信息冗余
 - Suppose a **relationship** set *sec_course* between entity sets **section** and **course**.
 - The information in *sec_course* is **redundant**, since **section** already has attribute *course_id*, which identifies the course with which the section is related.
 - To deal with **redundancy** is to get rid of the relationship *sec_course*; However, relationship between **section** and **course** becomes **implicit**.



Weak Entity Sets

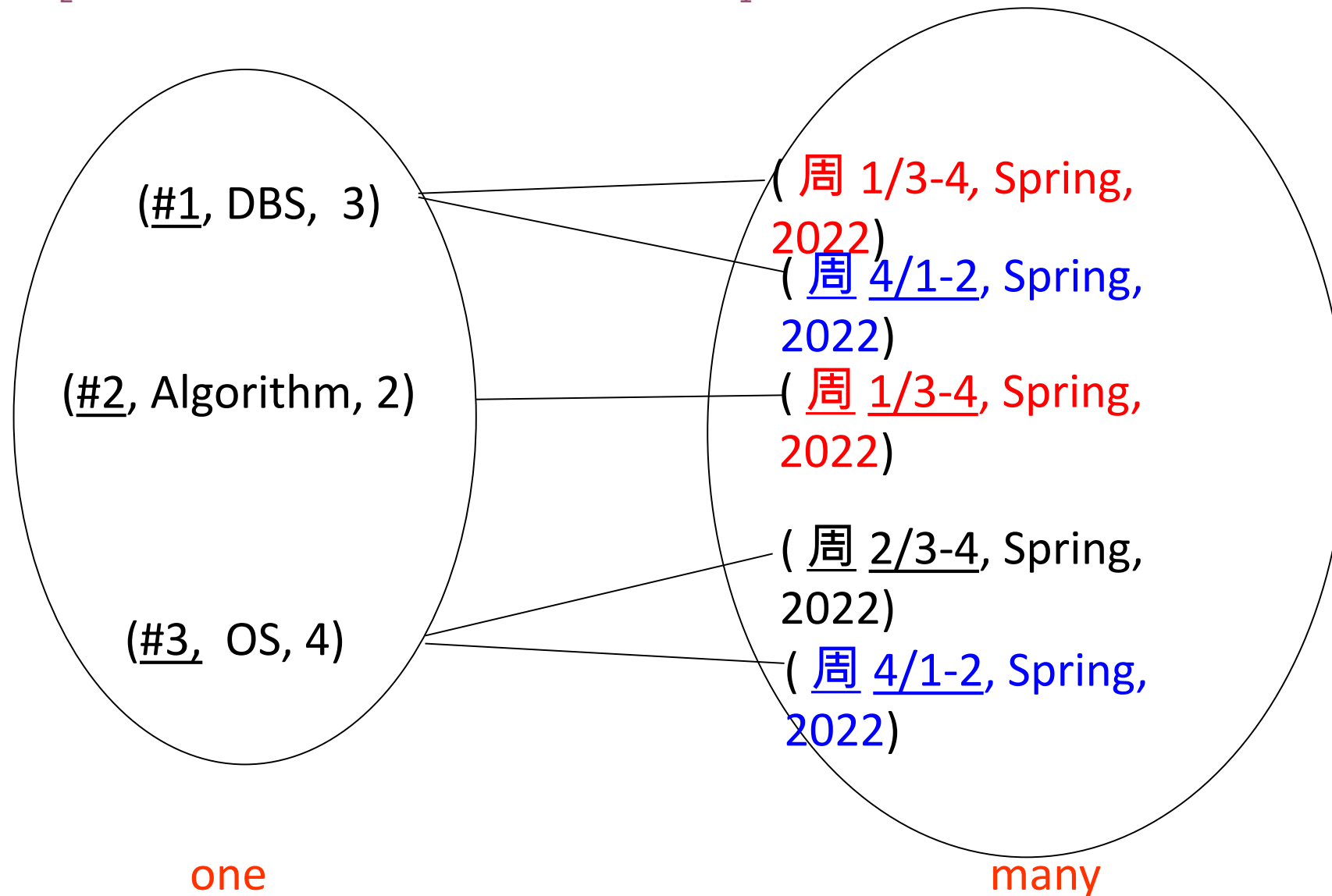
■ 方案 2 : 引入 weak entity set

- **Not** store the attribute *course_id* in the *section* entity
But only store the remaining attributes *section id*, *year*, and *semester*.
- However, the entity set *section* does **not** have enough attributes to identify a particular *section* entity **uniquely**
- To deal with this problem, we treat relationship *sec_course* as a **special relationship** that provides **extra information**
- *section* is modeled as a **weak entity set**



E₂: *course*

E₁: *section*/course-offering(id, semester, year)



Weak Entity Sets

- **Def:** **Identifying entity** of a **weak entity set**
Existence of **weak entity set** is dependent on **Identifying entity**.
- **Note** : Use **identifying entity**, along with extra attributes called **discriminator** to uniquely identify a weak entity, instead of a primary key
 - **primary key** of **identifying entity** + **discriminator** in weak entity set
 - e.g. **course_id** + *section_id, year, semester*
- **注释** : Every weak entity must be associated with an **identifying entity**;
weak entity set is **existence dependent** on identifying entity set.
 - The identifying entity set is to own the weak entity set.
- **Def:** **identifying relationship** is the relationship associating the weak entity set with the identifying entity set.

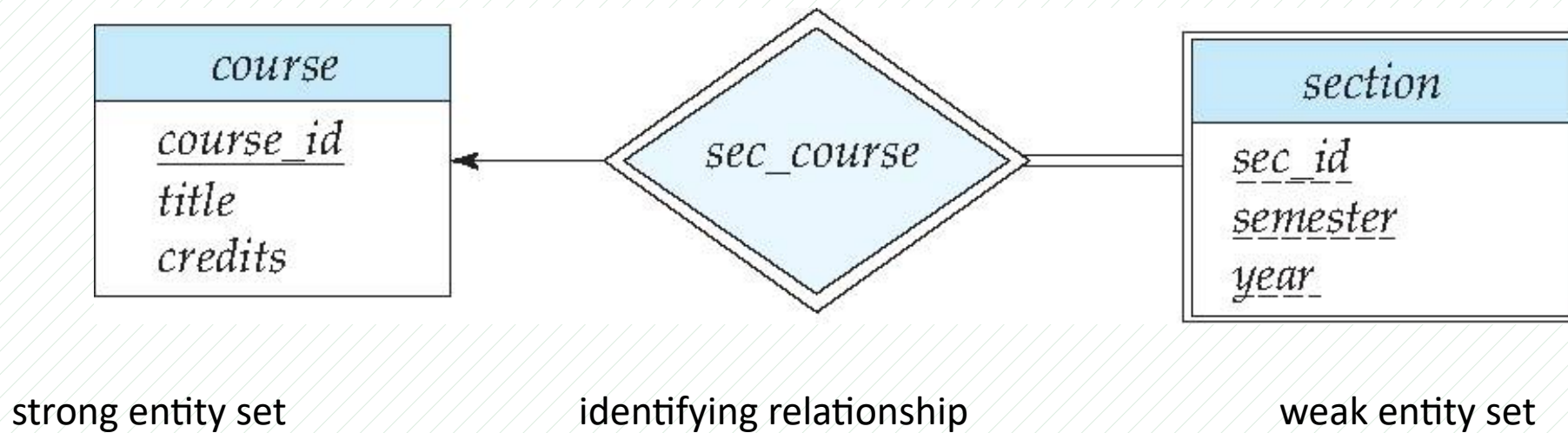
Expressing Weak Entity Sets

- **Note** : a **weak entity set** is depicted via a **double rectangle**.

Underline the **discriminator** of a weak entity set with a **dashed line**.

The **relationship set** connecting weak entity set to identifying strong entity set is depicted by a **double diamond**.

- Primary key for *section* – (*course_id*, *sec_id*, *semester*, *year*)



Expressing Weak Entity Sets

- **Def:** The **discriminator** (or **partial key**) of a weak entity set is the set of attributes that **distinguishes** among all those entities in weak entity set that depending on a **particular strong entity**
 - e.g. *sec_id, semester, year*
 - the **discriminator** of a weak entity set is underlined with a **dashed line**

Expressing Weak Entity Sets

- The **primary key of a weak entity set** is formed by
the **primary key of strong entity set** + the **discriminator of weak entity set**
 - the **primary key for weak entity set** E_1
= discriminator $\cup$ $\text{primary_key}(E_2)$.
 - e.g. sec_id, semester, year, *course_id*
- Weak entity set E_1 with several identifying entity sets $\{E_2, E_3, \dots, E_n\}$
the **primary key for weak entity set**
 $E_1 = \text{discriminator} \cup \text{primary_key}(E_2) \cup \dots \cup \text{primary_key}(E_n)$.

6.6 Removing Redundant Attributes

- Suppose we have entity sets:
 - **student**, with attributes: *ID*, *name*, *tot_cred*, **dept_name**
 - **department**, with attributes: **dept_name**, *building*, *budget*
- Each student has an associated department using a **relationship** set *stud_dept*
- The attribute **dept_name** in *student* replicates information present in relationship
 - It is **redundant**
 - It needs to be **removed**.
- **BUT**: when converting back to tables, the attribute should be reintroduced.

6.7 Reducing E-R Diagrams to Relation Schemas

- **Logical database design** with two steps:
 - **generation** of the initial relational schema from the E-R diagram,
 - **normalization** of relational schema (Chapter 7)
- **Converting an E-R diagram to a table format** is the basis for **deriving a relational database design**
- A database which conforms to an **E-R diagram** can be represented by a collection of **schemas/tables**
 - **entity sets** and **relationship sets** can be expressed uniformly as **relation schemas**.
 - There is a **unique schema** corresponding to **entity set** or **relationship set**.

Representing Entity Sets

- A **strong entity set** reduces to a schema with the **same attributes**

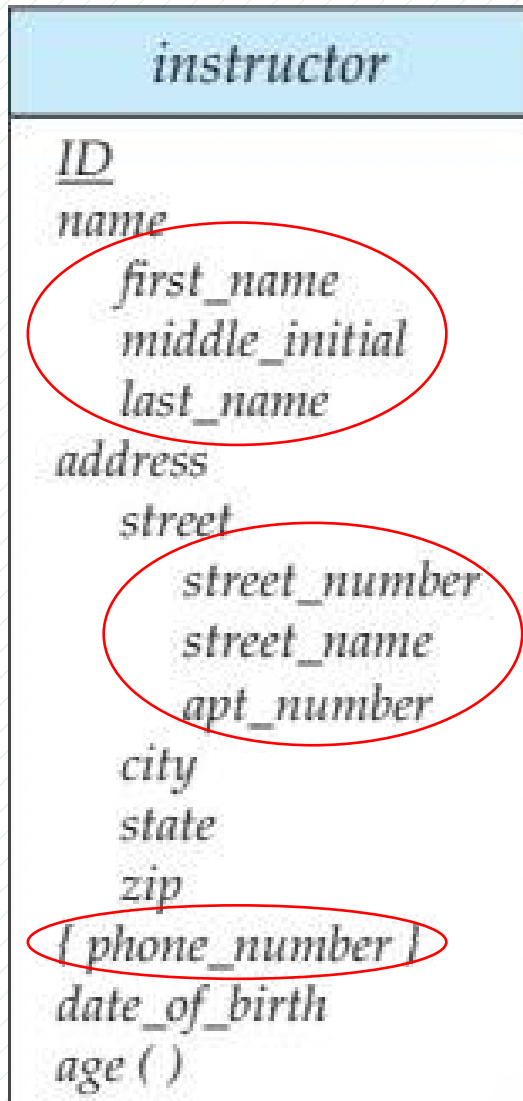
student (**ID**, name, tot_cred)

- A **weak entity set** becomes a table that **includes a column for the primary key** of the identifying **strong entity set**

section (**course_id**, sec_id, sem, year)



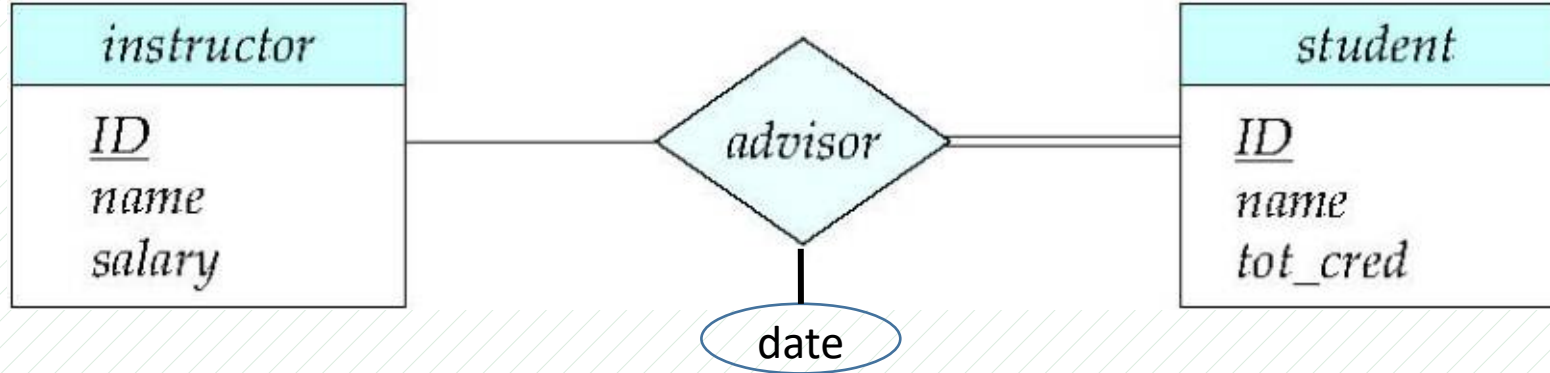
Representation of Entity Sets with Composite Attributes



- **Composite attributes** are flattened by **separate attributes**
- an entity set *instructor* with composite attribute **name**
 - the schema corresponding to the entity set has attributes **name_first_name** and **name_last_name**
 - Prefix omitted if there is no ambiguity
 - (*name_first_name* could be *first_name*)
 - Ignoring multivalued attributes,
- Extended instructor schema is given by
 - instructor(ID, **first_name, middle_initial, last_name, street_number, street_name, apt_number, city, state, zip_code, date_of_birth, age**)

Representing Relationship Sets

- **Reduction** of a **relationship set** into **tables** are strongly dependent on the **mapping cardinality** constraint and **total/partial constraints**
- 1. A **many-to-many** relationship set is represented as a table with columns for the **primary keys** of two participating entity sets, **and** any **descriptive attributes** of the relationship set



instructor(*ID*, *name*, *salary*)

student(*ID*, *name*, *tot_cred*)

***advisor*(*instructor.ID*, *student.ID*, *date*)**

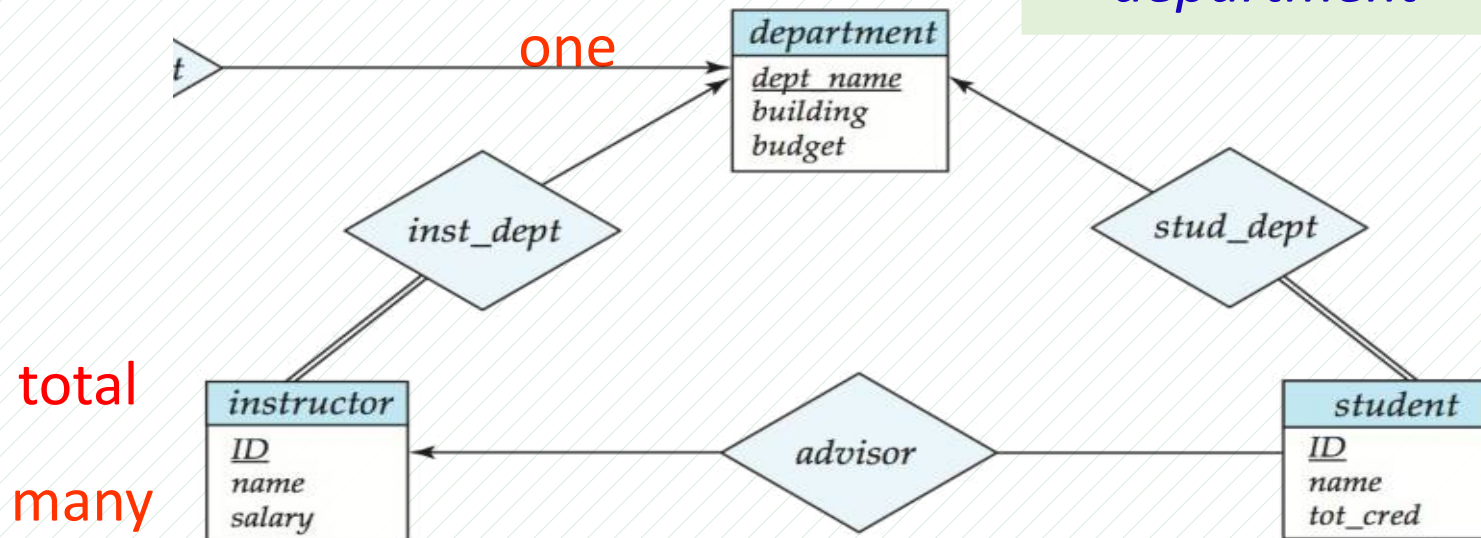
Many-to-Many

- For relationship set R between entity set A and B ,
primary-key(A) $\cup$ primary-key(B) = $\{a_1, a_2, \dots, a_m\}$,
descriptive attributes of R are $\{b_1, b_2, \dots, b_n\}$
- Then R can be reduced to
 - table R with one column for each attribute in
 $\{a_1, a_2, \dots, a_m\} \cup \{b_1, b_2, \dots, b_n\}$

Many-to-one/One-to-many

- 2. **Many-to-one** and **one-to-many** relationship sets that **total on the many-side** can be represented by adding an extra attribute to the **many** side, containing the primary key of the **one** side
- ie: Instead of creating a schema for relationship set *inst_dept*, add an attribute *dept_name* to the schema arising from entity set *instructor*
 - *instructor* (ID, name, salary, *dept_name*)
 - *department* (dept_name, building, budget)

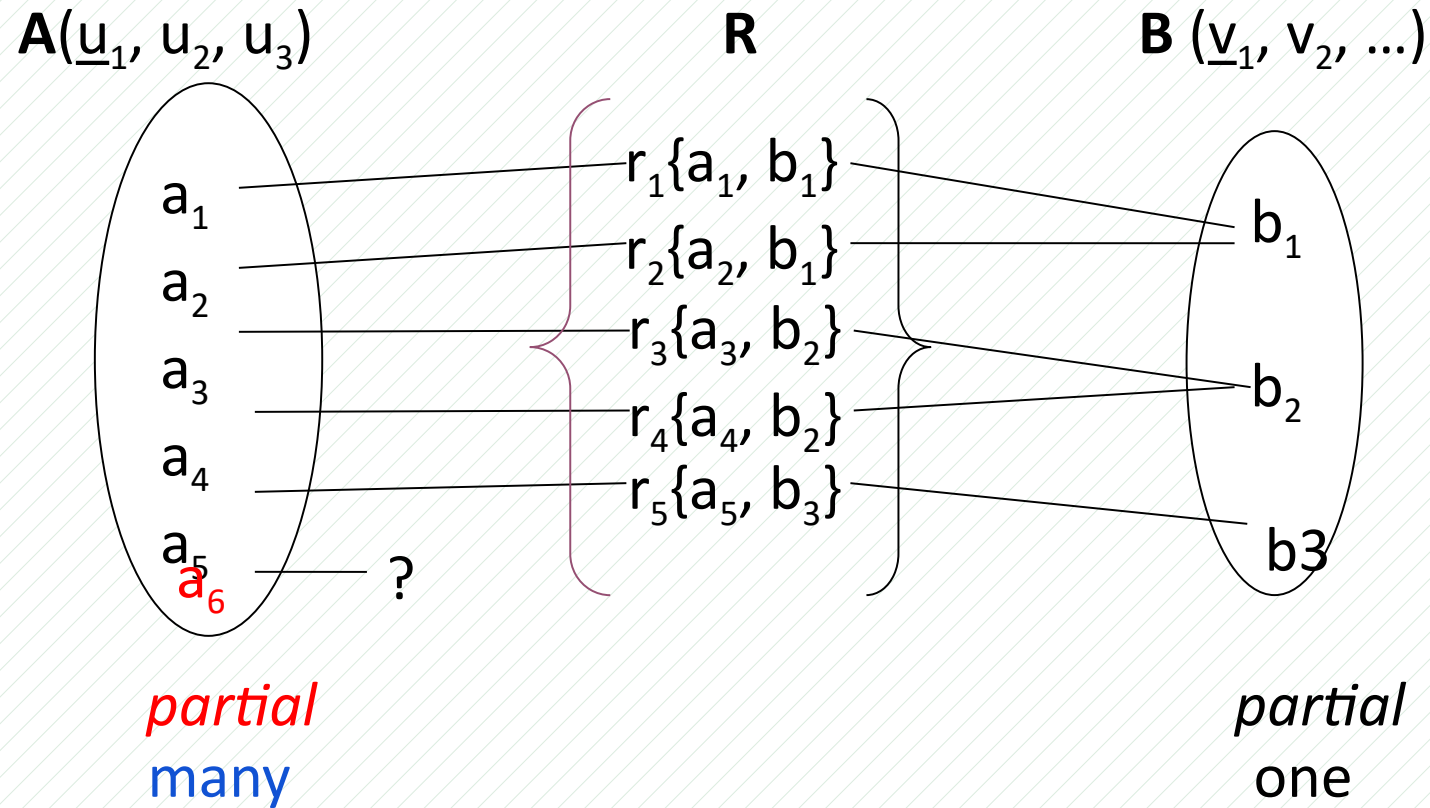
▫ foreign key referencing constraint between *instructor* and *department*



Many-to-one Relationship Sets with partial participation

- 3. Many-to-one relationship sets with partial participation on the many side
- **The relationship** is reduced to an **independent** table, avoiding **null** values
 - **联系单独转为一张表**
- ie, 上图中联系 *advisor*, from *student* to *instructor*, 单独转为关系表
advisor (instructor.id, student.id)
- If participation is partial on the many side, replacing a schema by an extra attribute in schema corresponding to many side, could result in null values
 - 联系不单独转表，而是归入多端实体对应的表中，将会在多端实体表中引入空值

Many-to-one Relationship Sets with partial participation



Why not reducing R into table A

Many-to-one Relationship Sets with partial participation

- If $R = \{r_1, r_2, r_3, r_4, r_5\}$ is reduced to table

$$A'(\underline{u}_1, u_2, u_3, v_1) = \{a_1', a_2', a_3', a_4', a_5', a_6'\},$$

- ▣ a_6 does not participate in R
- ▣ thus, a_6' will have *null* on the attribute v_1

Table $A' = \{a_1', a_2', a_3', a_4', a_5', a_6'\}$

	$\underline{u}_1$	$\underline{u}_2$	$\underline{u}_3$	$\underline{v}_1$	
a_1'	*	*	*	*	$r_1\{a_1, b_1\}$
a_2'	*	*	*	*	$r_2\{a_2, b_1\}$
a_3'	*	*	*	*	$r_3\{a_3, b_2\}$
a_4'	*	*	*	*	$r_4\{a_4, b_2\}$
a_5'	*	*	*	*	$r_5\{a_5, b_3\}$
a_6'	*	*	*	<i>null</i>	

Why not reducing R into table A

Reducing One-to-one Relationship Sets

- 4. For one-to-one relationship sets,
 - either side can be chosen to act as the **many** side
 - ▣ an extra attribute can be added to either of the tables corresponding to the two entity sets

6.8 Extended E-R Features

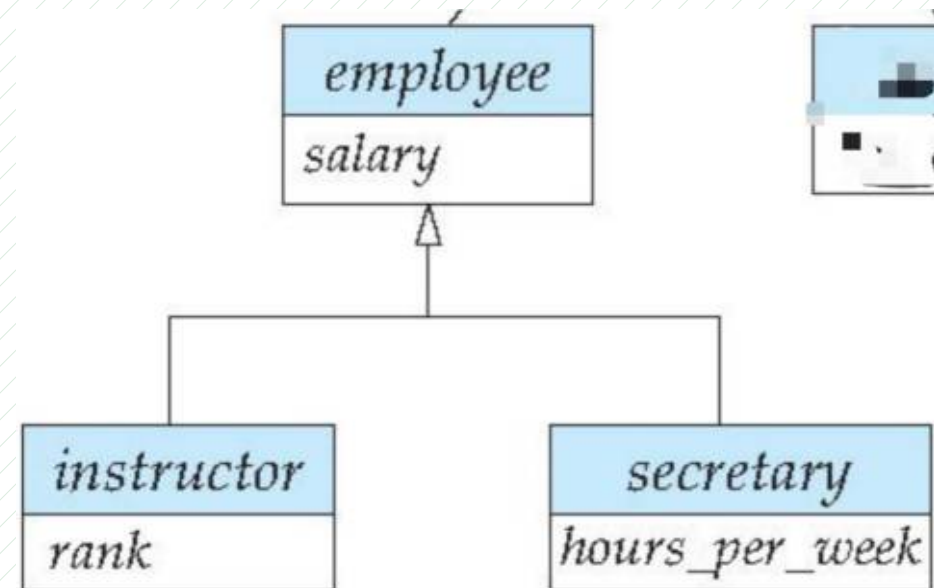
- **Object-oriented** (面向对象) E-R
 - **specialization** (特化 , 特殊化)
 - **generalization** (泛化 , 普遍化)
- **Attributes inheritance** (属性继承)

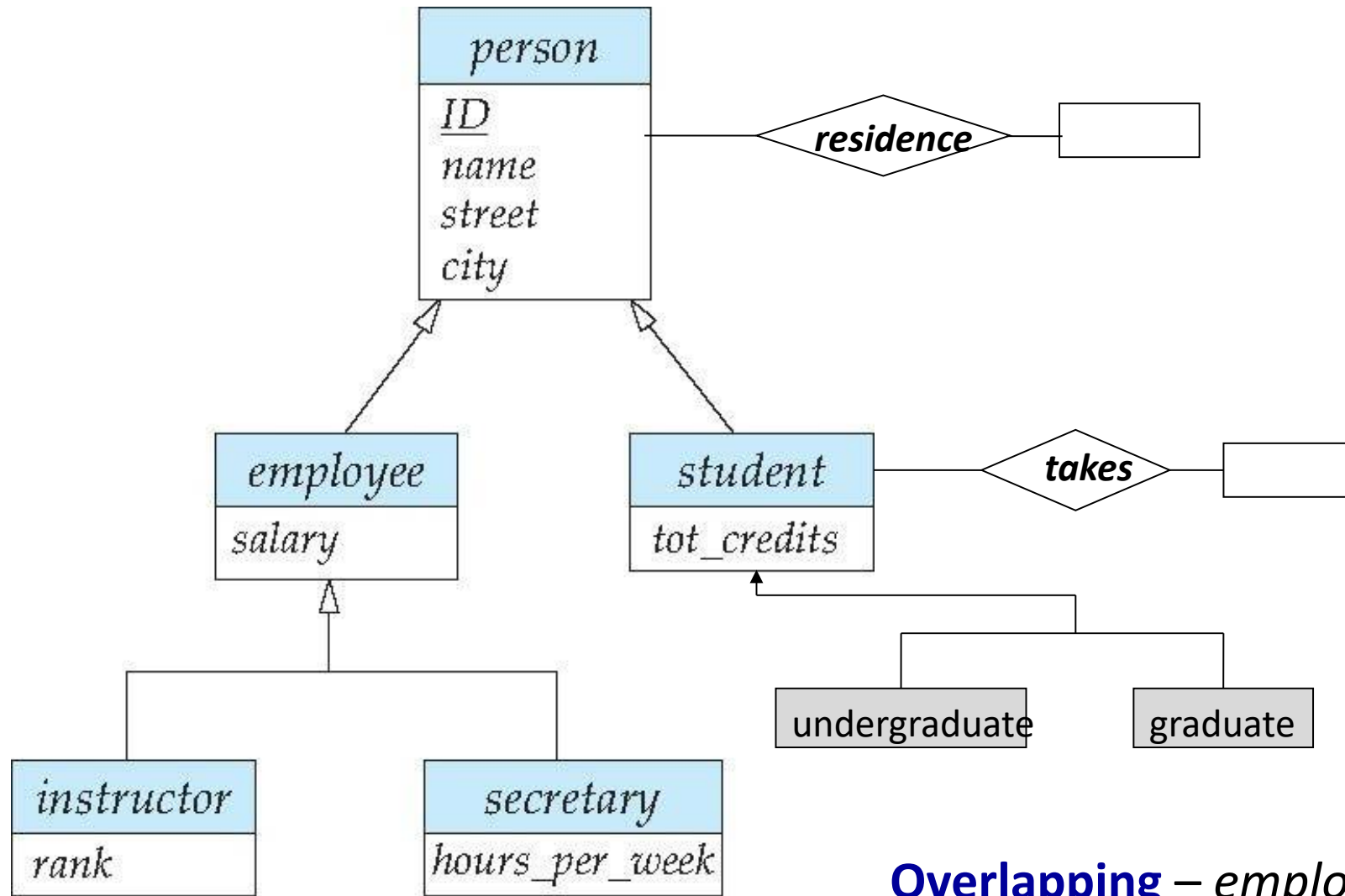
1. Specialization

- **Def: Top-down design process:**

We designate **sub-groupings** within an entity set that are distinctive from other entities

- ▣ Sub-groupings become **lower-level** entity sets
- ▣ Attributes **specific** to **lower level** entity sets
- ▣ Attributes **apply** to the **higher-level** entity set
- ▣ Depicted by a **triangle** component labeled ISA
- ▣ e.g., *instructor* **is an** *employee*





Overlapping – *employee* and *student*

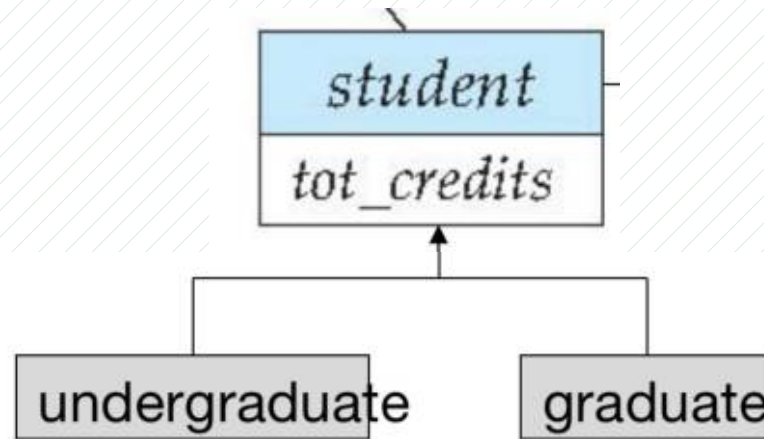
Disjoint – *instructor* and *secretary*

Fig. 6.18 Specialization and generalization

1. Specialization

■ Attribute inheritance

- a **lower-level** entity set **inherits** all **attributes** of the **higher-level** entity set
- a **higher-level** attributes apply to all its **lower-level** entities



2. Generalization

- **Def: A bottom-up design process**
 - **Combining** a number of entity sets that share the same features into a **higher-level** entity set.
- Specialization and generalization are **inversions** of each other
 - they are represented in E-R diagram in **the same way**.
 - specialization and generalization are used **interchangeably**.

Superclass vs Subclass

- **Note:** The **ISA** relationship is also referred to as **superclass-subclass** relationship
- **Superclass and subclass**
 - As for the entity sets A and B
 - if A is the generalization of B , B is the specialization of A ,
 - If A is the superclass of B , B is the subclass of A

Constraints on Generation/Specialization

- **Constraints:** **Determine** the **membership** of lower-level entities
- **Divide** **higher-level** entity set **into** **lower-level** entity sets
 - ▣ **Condition-defined, attribute-defined**
 - based on the condition/predicate/attributes-value
e.g. *student_type*= graduate, undergraduate
 - ▣ **User-defined**
 - e.g. all instructors are divided into four teams
based on their assigned works
 - each team becomes a subclass of the entity set *instructor*

Completeness Constraint

■ **Note** : Completeness constraint

- whether an entity in the **higher-level** entity set must **belong to** at least one of the **lower-level** entity sets
- ▣ **total**: an entity **must** belong to one of the lower-level entity sets
 - e.g. *student*, all entities must be either graduate or undergraduate
- ▣ **partial**: an entity **need not** belong to one of the lower-level entity sets
 - e.g. *person* specialization

Completeness Constraint

- **Specify total generalization**
- Adding keyword **total** in diagram and drawing a dashed line from the keyword **to** the corresponding hollow arrow-head,
 - ▣ the set of hollow arrow-heads

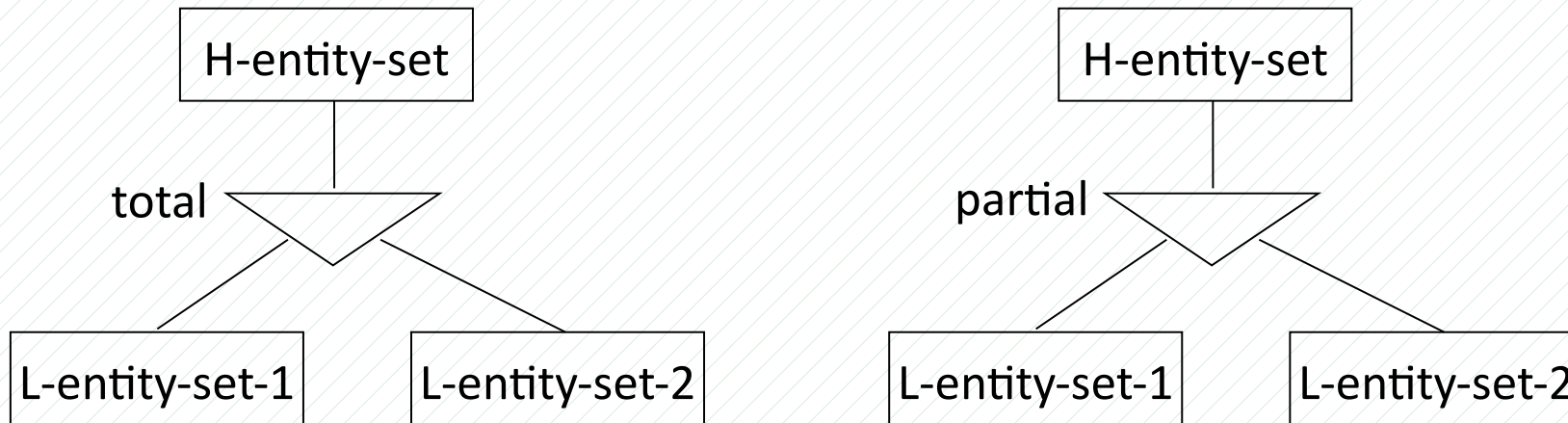
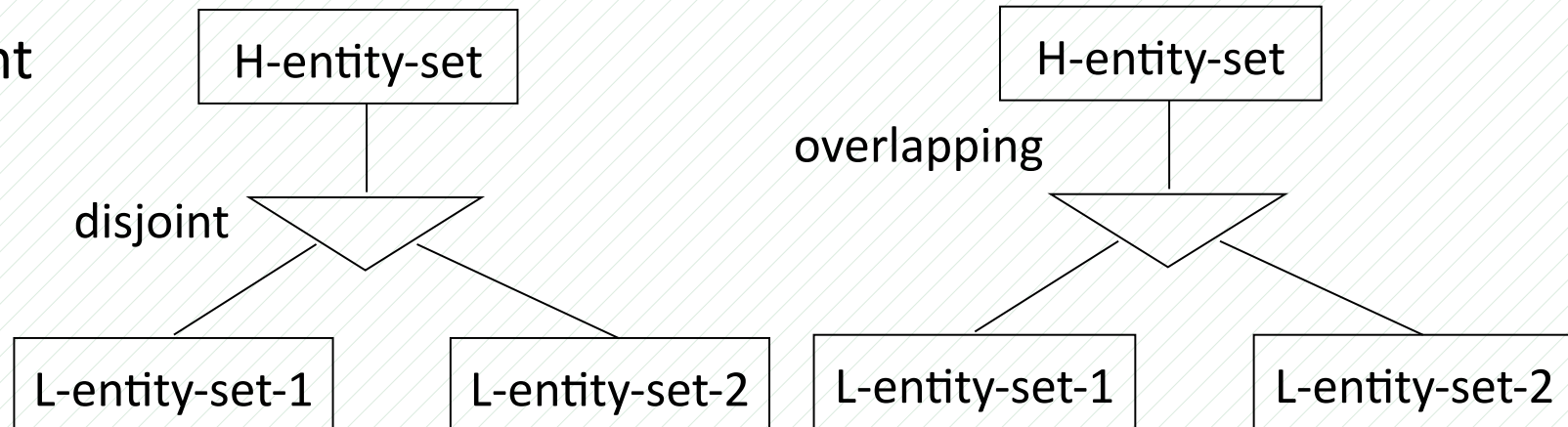


Fig.6.0.12 Completeness constraints

Completeness Constraint

- **Constraints** on whether a **high-level** entity may **belong to** more than one **lower-level** entity set
 - ▣ **1. disjoint**
 - a high-level entity can belong to only one lower-level entity set, i.e., $L\text{-entity-set-1} \cap L\text{-entity-set-2} = \Phi$
 - e.g. instructor and secretary
 - ▣ **2. overlapping**
 - a high-level entity can belong to more than one lower-level entity sets
 - $L\text{-entity-set-1} \cap L\text{-entity-set-2} \neq \Phi$
 - e.g. employee and student



Representing Specialization via Schemas

■ Method 1

- **Form** a schema for the **higher-level** entity
- **Form** a schema for each **lower-level** entity set, **include** primary key of **higher-level** entity set and local attributes

schema	attributes
person	ID, name, street, city
student	ID, tot_cred
employee	ID, salary

■ Drawback

- an employee requires accessing **two relations** to get information one corresponding to the **low-level** schema **and** the other one corresponding to the **high-level** schema

Representing Specialization via Schemas

■ Method 2

- **form** a schema for each entity set with all local and inherited attributes

schema	attributes
person	ID, name, street, city
student	ID, name, street, city, tot_cred
employee	ID, name, street, city, salary

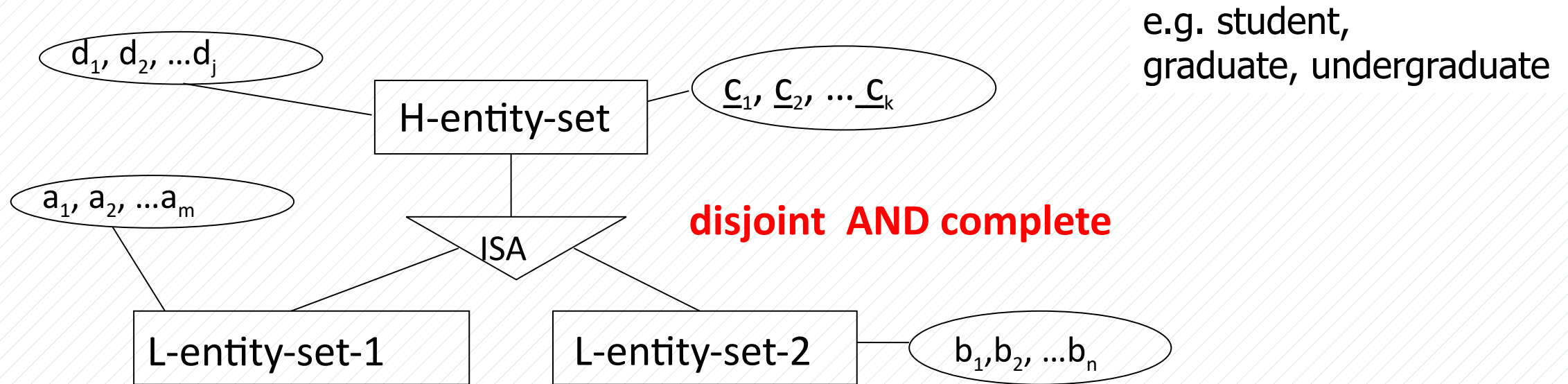
■ Drawback

- *name*, *street* and *city* may be stored **redundantly** for people who are both students and employees

Representing Specialization via Schemas

■ Method 3

- if generalization/specialization is **disjoint** and **total**, creating two tables



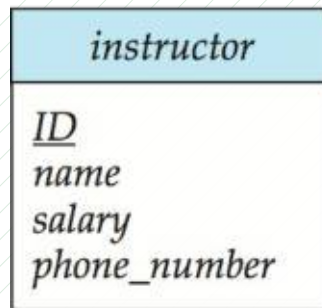
- $T_2(a_1, a_2, \dots, a_m, c_1, c_2, \dots, c_k, d_1, d_2, \dots, d_j)$ for L-entity-set-1
- $T_3(b_1, b_2, \dots, b_n, c_1, c_2, \dots, c_k, d_1, d_2, \dots, d_j)$ for L-entity-set-2
- **Replacing** high-level entity set H-entity-set **with** T_2 and T_3 .

6.9 Issues in Entity-Relationship Design

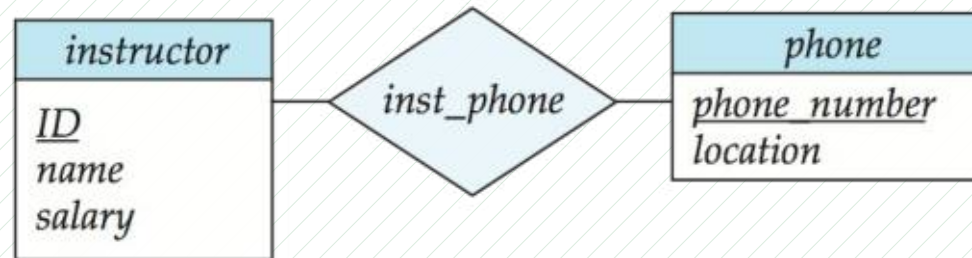
- **1. Mistakes** in E-R Diagrams
- **2. Entities** vs. **Attributes**
- **3. Entities** sets vs. **Relationship** sets

1. Use of entity sets vs. attributes

- **Mapping** “thing/object” into **entity** sets or **attribute** sets
- e.g. the instructor and phone **objects** in real world
 - ▣ **model 1.** entity: *instructor* (*ID*, *name*, *salary*, *phone-number*)
 - implying that employees have precisely one phone number each,
 - **on** less phone-number is multivalued
 - how to represent properties of an object **phone** ?



(a) model1

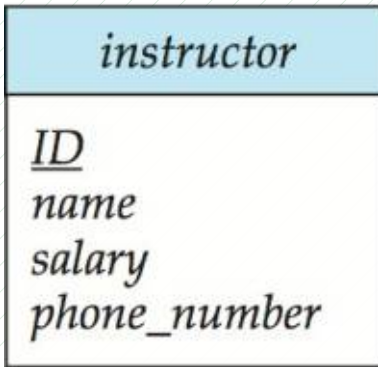


(b) model2

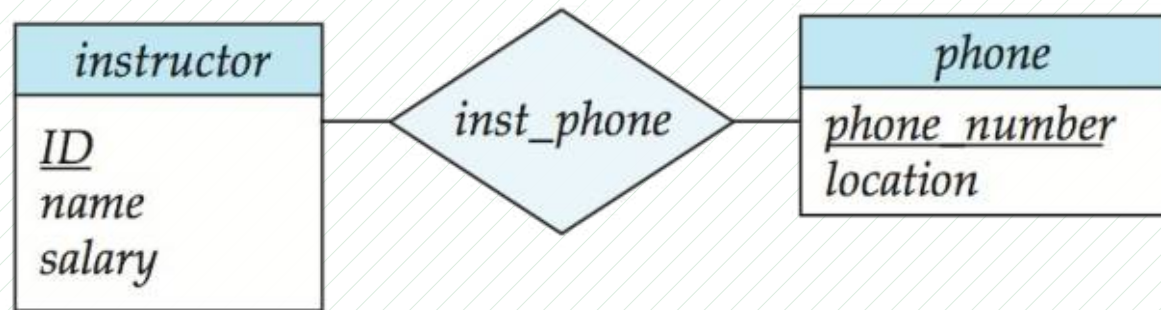
1. Use of entity sets vs. attributes

□ model 2

- entity: *instructor* (*ID*, *name*, *salary*)
phone (*phone-number*, *location*)
- relationship: *inst-phone*



(a) model1

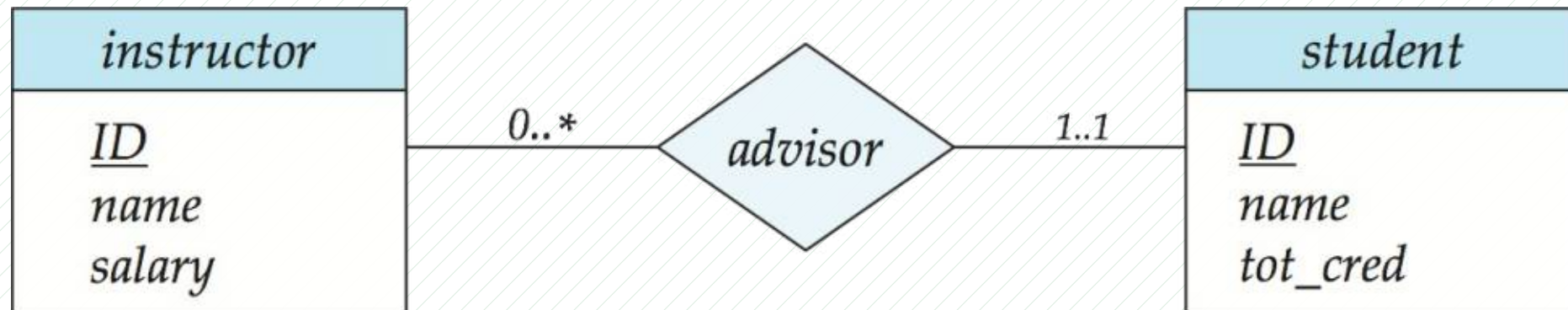


(b) model2

- Treating *phone* as an entity, more information can be modeled, e.g.
 - location
 - multiple phone numbers

2. Use of entity sets vs attributes

- **Rule1.** Not use the **primary-key** of an entity set as attributes of **another** entity set.
it is better to use an **relationship set** to **explicitly** show this **association**
 - e.g. for entity *student*, not use *instructor.id* as its attribute, in stead of, representing association among *student* and *instructor* using relationship *advisor*



2. Use of entity sets vs attributes

- **Rule2.** If *E* participate in *R*, **not to designate** the primary-key(*E*) as attributes of *R*, to avoid **information redundancy**
 - i.e., an *ill* modeling of *advisor*

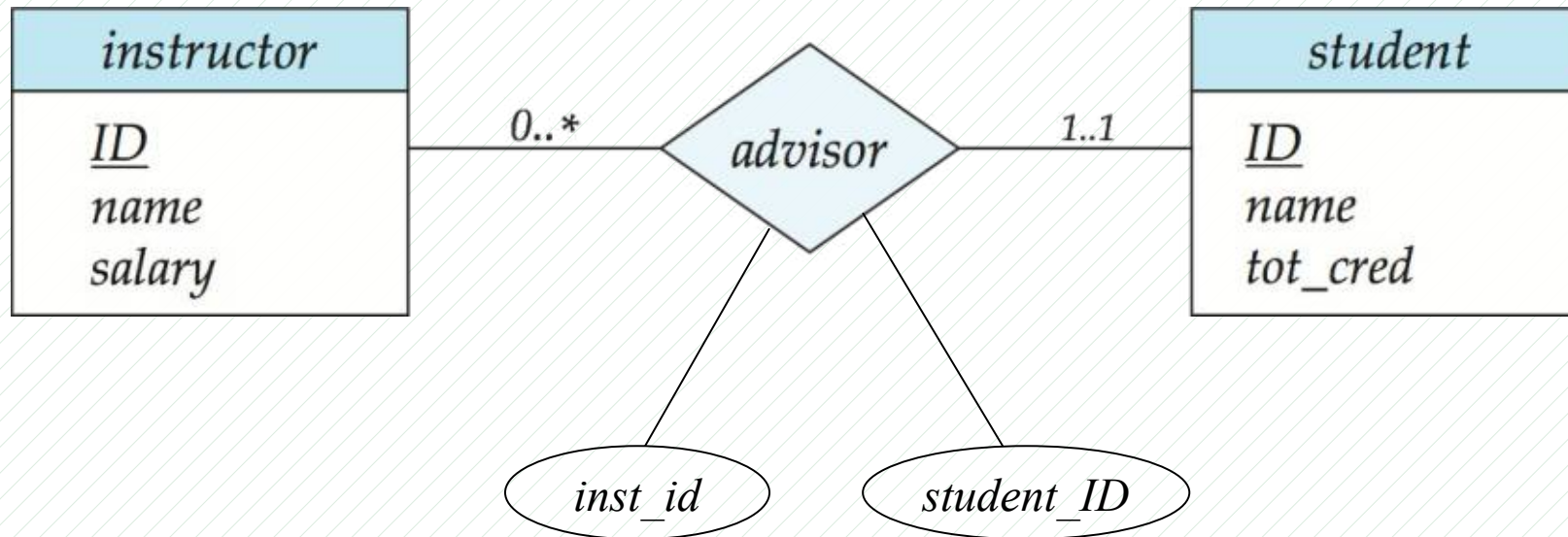
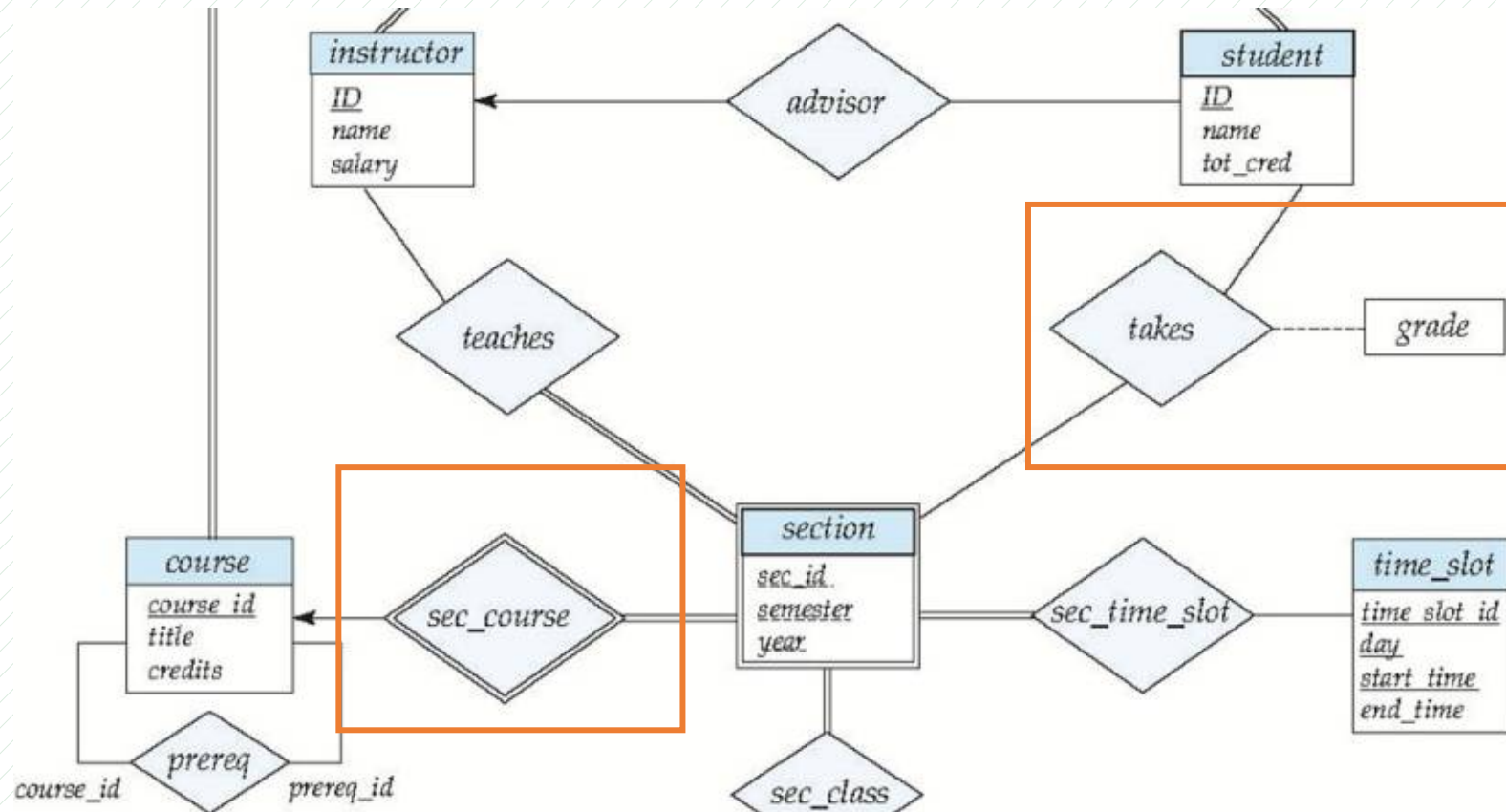


Fig.6.0.7 Data redundancy in *advisor*

2. Use of Entity Sets vs Relationship Sets

- **Rule 3.** Possible guideline is to **designate** a **relationship** set to describe an action that occurs between entities
 - e.g. The relationship *takes* is used to model a situation where *a student* takes *a section* of *a course*



3. Binary Vs. Non-Binary Relationships

- **Rule4.** Using the **binary** relationship set, replace a **non-binary** relationship set by a number of **binary** relationship set
 - e.g. , a ternary relationship *parents*, relating a child to his/her father and mother, is best replaced by two binary relationships, *father* and *mother*
 - using three binary relationships allows partial information

Converting Non-Binary Relationships to Binary Form

- For 3-ary relationship set $R = \{(a_i, b_i, c_i) \mid a_i \in A, b_i \in B, c_i \in C\}$ among entity sets A, B, C , i.e. $R \subseteq A \times B \times C$.
Replace R by an entity set $E = \{e_i\}$ and three binary relationship set R_a, R_b, R_c

Converting Non-Binary Relationships to Binary Form

- $E = R = \{ e_i \} = \{ (a_i, b_i, c_i) \}$, i.e. each (a_i, b_i, c_i) corresponds to one e_i in E ,
- $R_a = \{ (e_i, a_i) \mid e_i \in E, a_i \in A \}$, relating E and A
- $R_b = \{ (e_i, b_i) \mid e_i \in E, b_i \in B \}$, relating E and B
- $R_c = \{ (e_i, c_i) \mid e_i \in E, c_i \in C \}$, relating E and C
- E has an identifying attribute e (candidate key) to distinguish each e_i in E

4. Placement of Relationship Attributes

- **Problem:** The attributes of relationship sets ***R*** may be associated with one of its participating entity sets
- For the attribute ***attr-R*** in relationship set ***R*** between E_1 and E_2 ($R \subseteq E_1 \times E_2$), if the cardinality of ***R*** is
 - *one-to-one*
 - ***attr-R*** can be assigned as attribute of either E_1 or E_2 , rather than relationship set ***R***
 - *one-to-many* from E_1 to E_2
 - ***attr-R*** can only be assigned as the attribute of the entity set E_2 (*many-side*), rather than the relationship set ***R***

Placement of Relationship Attributes

- **Example.** the relationship *advisor* from *instructor* to *student* is *one to many*, the attribute *date* of *advisor*, when an instructor became the advisor of the student, can be made as an attribute of *student*

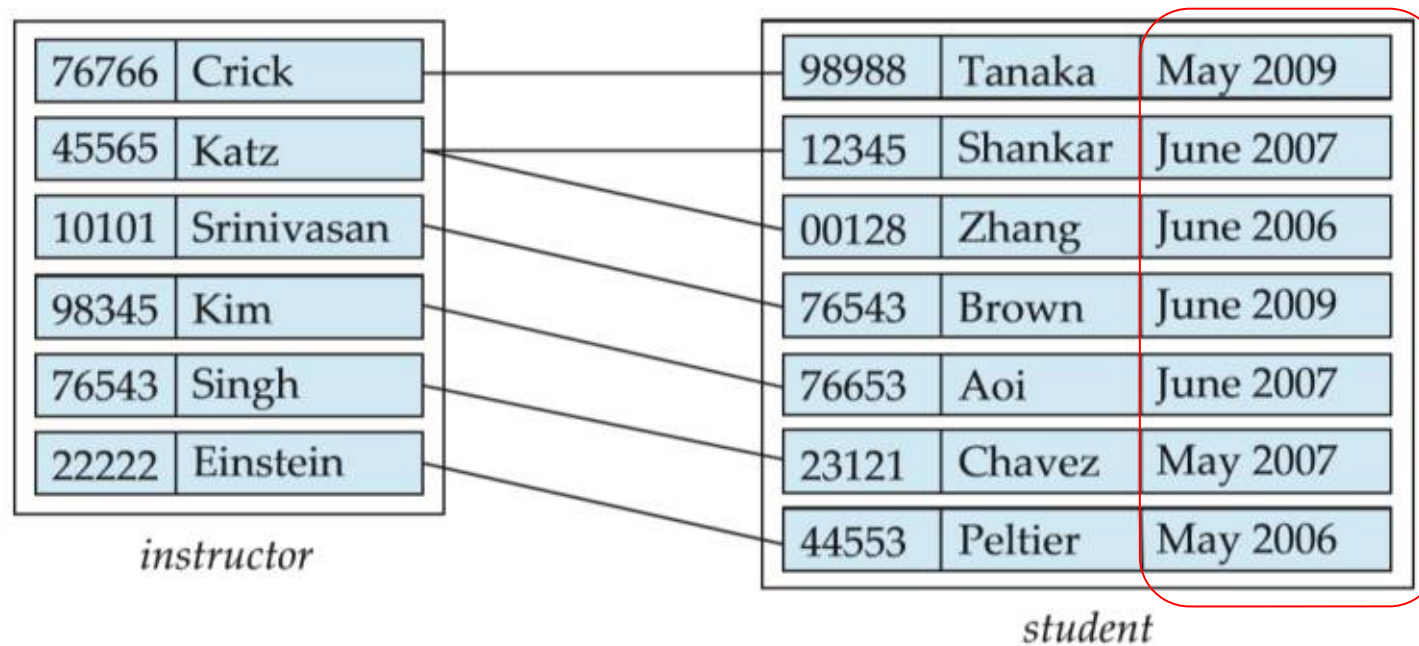


Figure 7.20 *date* as an attribute of the *student* entity set.

Placement of Relationship Attributes

- *Many-to-many*
 - ▣ **attr-*R*** may only be assigned as the attribute of *R*,
not of its participating entity sets *E*₁ or *E*₂

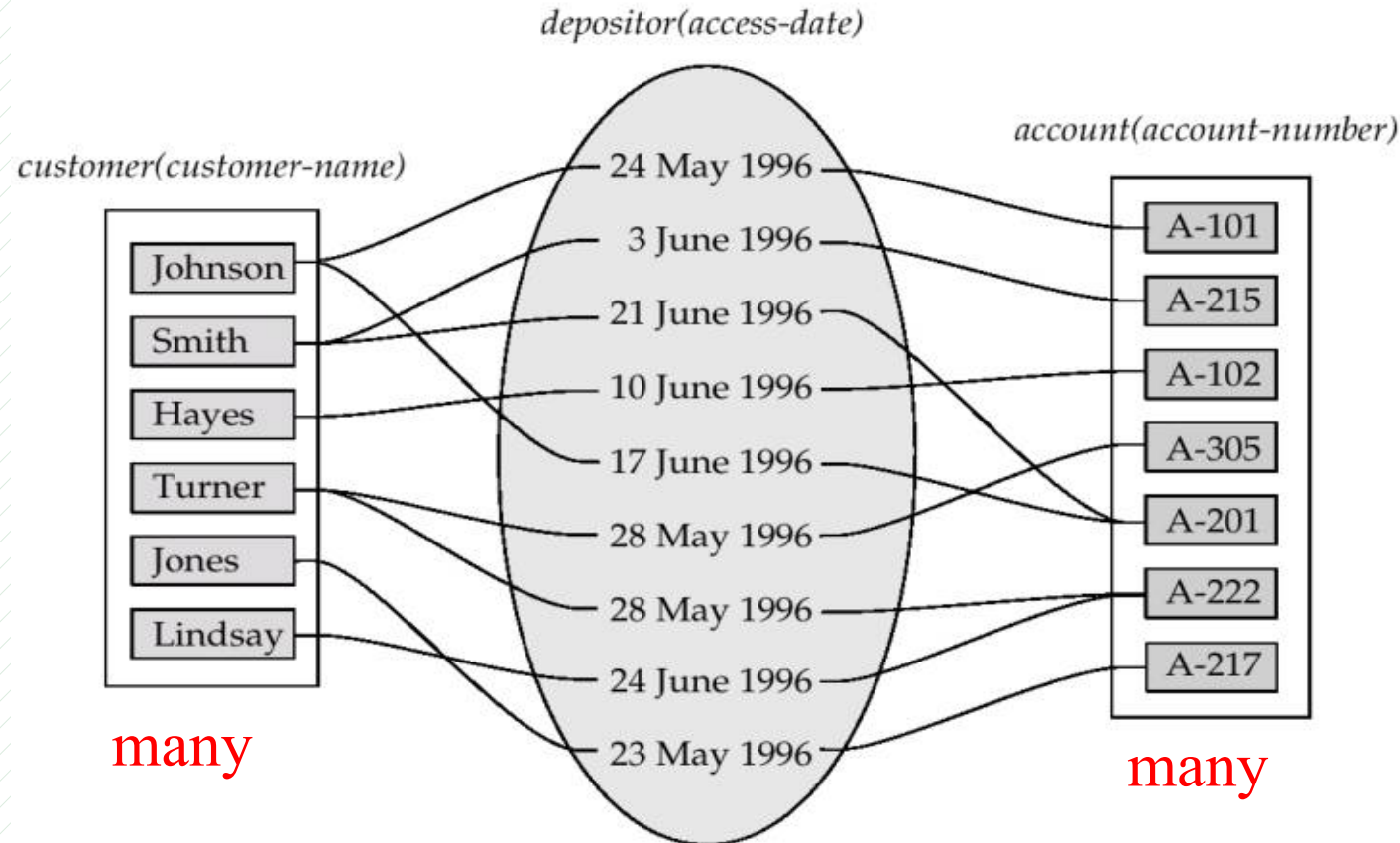
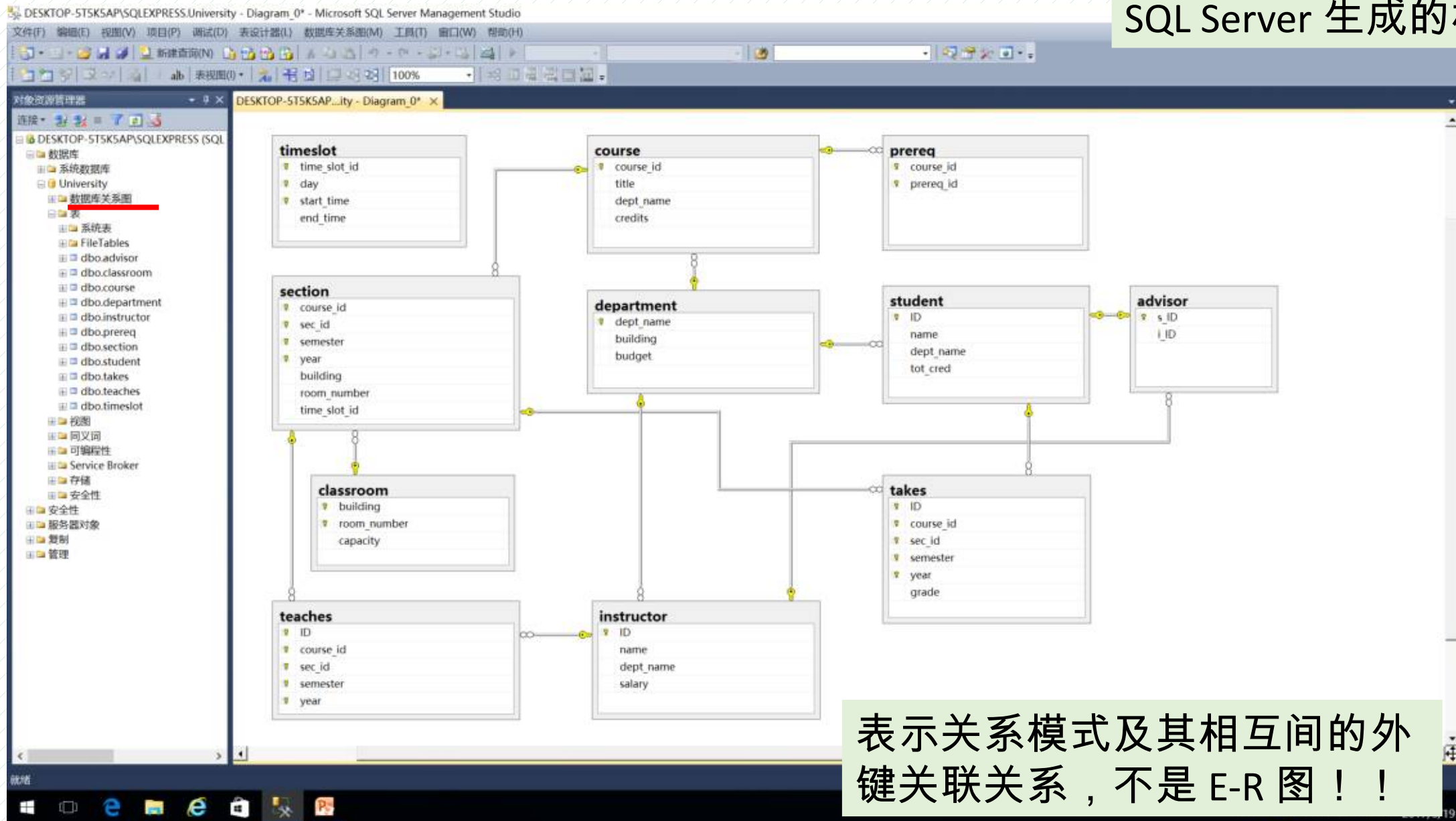


Figure *Access-date* as attribute of the *depositor* relationship set

Schema Diagram in Relational Model

SQL Server 生成的模式图



表示关系模式及其相互间的外键关联关系，不是 E-R 图！！

Conclusions

- 1. The use of an **attribute** or **entity** set to represent an object.
- 2. A real-world concept is best **expressed** by an entity set or a relationship set.
- 3. The use of a **ternary** relationship versus **a pair of binary** relationships.
- 4. The use of a **strong** or **weak** entity set.
- 5. The use of **specialization/generalization**.

Appendix 6-1 例题 1

- **Problem:** E-R modeling and table reductions in Ordering database
- A database used in an ordering system contains information about *customers*, *products* and *orders*. The following information is to be included:
 - ▣ for each **customer**
 - *customer-number*,
 - ship-to-addresses* (several per customer),
 - name*, *balance*, *discount*
 - ▣ for each **order** (订单)
 - *order number*, *customer number*, *ship-to address* (one per order), *date of ordering* (including *year*, *month*, and *day*),
 - products* (several kinds per order),
 - quantity* ordered of each product

Appendix 6-1 例题 1

- for each kind of **product**
 - *product-number, name, manufacturing-plant, quantity on hand*
- Entity sets: **Customer, Order, Product** and their *attributes*
 - In **Customer**, *ship-to addresses* is a multi-valued attribute, but in **Order**, *ship-to addresses* is a single-valued attribute
 - In **Order**, *date* is a composite attribute
- Some constraints(i.e., integrity constraints) about this ordering system
 - a customer may *make* one or more orders, and some customers may make no orders
 - each order must be for one (and only one) customer
 - *relationship sets*
 - relationship set *make* between **Customer** and **Order**
 - **Customer** partly participate *make*, and **Order** totally participate *make*
 - in relationship set *make*, mapping cardinality from **Customer** to **Order** is one-to-many

Appendix 6-1 例题 1

- each order must *contain* at least one kind of products
 - a kind of product may appear in several orders, but some kinds of products may not be ordered by any customers
 - *relationship set*
 - relationship set *contain* between *Order* and *Product*, and descriptive attributes *quantity of ordered* for relationship set *contain*
 - *Product* partly participate *contain*, and *Order* totally participate *contain*
 - in relationship set *contain*, mapping cardinality from *Order* to *Product* is many-to-many
- Problem: E-R diagram of the Ordering DB?

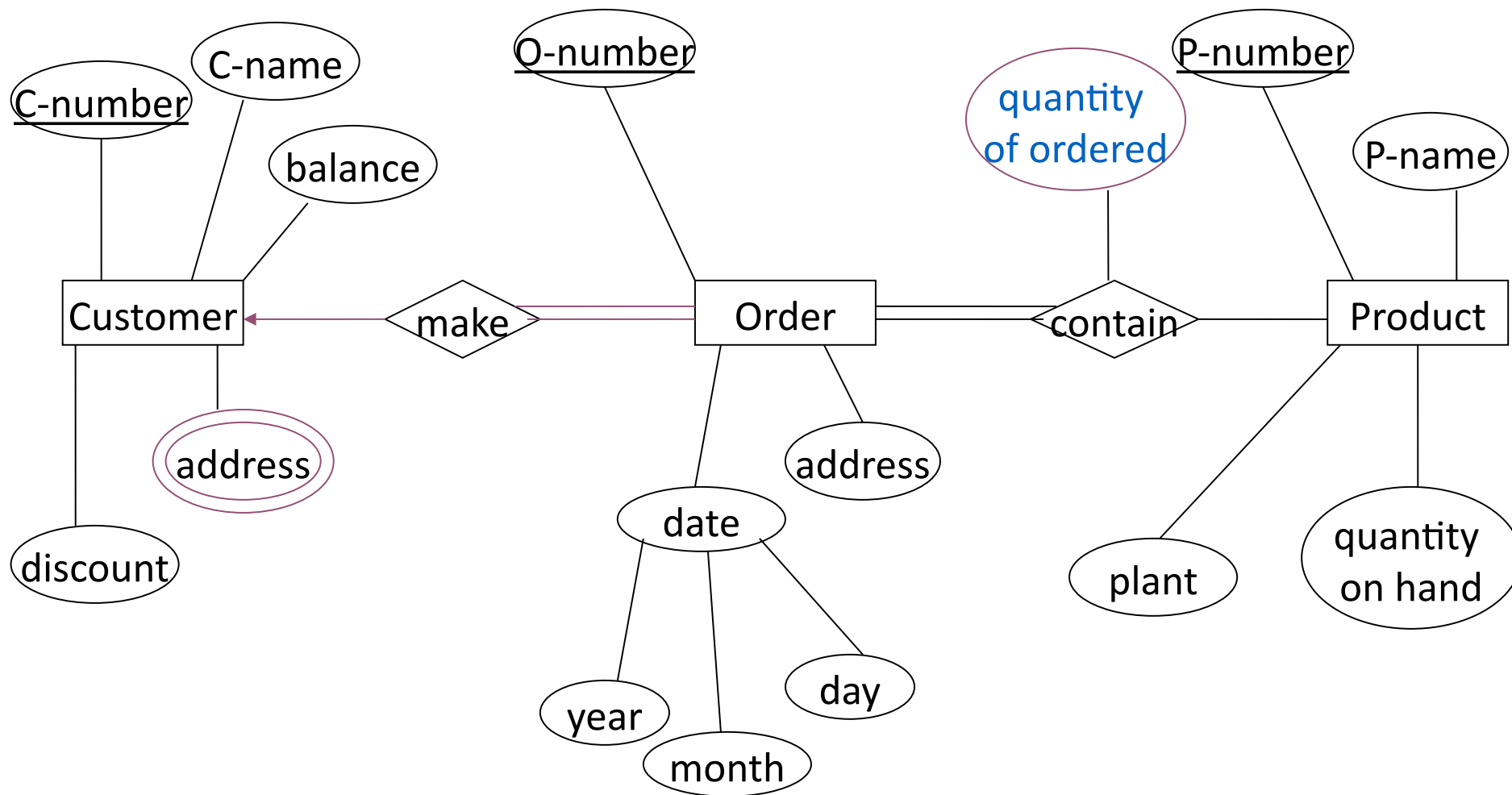


Fig. C.1. E-R diagram of the ordering DB is shown in

Appendix 6-1 Example 1

- The E-R diagram is then reduced to some relational tables
- For entity set **Customer** and its multi-valued attribute *ship-to addresses*, there are two tables
 - ▣ **customer**=(C-number, c-name, discount, balance)
 - ▣ **ship-to-addresses** = (C-number, ship-to-addresses)
- For entity set **Order** and its composite attribute **date** which should be decomposed , there is a table
 - ▣ **order1**=(O-number, ship-to-address, year, month, day)
- For entity set **Product**, there is a table
 - ▣ **product**=(P-number, p-name, plant, quantity-on-hand)

Appendix 6-1 Example 1

- For relationship set *make*, there are no descriptive attributes, mapping cardinality from *Customer* to *Order* is *one-to-many*, and *Order* totally participate *make*. So, *make* should not be reduced into a separate table, and can be represented by adding the primary key C-number of *Customer*, that is *C-number* into the table *order*
 - ▣ *order2*=(O-number, C-number, ship-to-address, year, month, day)
 - ▣ note: the primary key C-number is already in the table *order*

Appendix 6-1 Example 1

- Supposing mapping cardinality from *Order* to *Product* is many-to-many, for relationship set *contain*, there is a descriptive attribute *quantity of ordered*, *contain* should be represented as a separate table with columns for the primary keys of the two participating entity and descriptive attribute
 - ▣ *contain*=(O-number, P-number, quantity of ordered))
- Supposing mapping cardinality from *Order* to *Product* is one-to-many, because *Product* at the many side only *partly participate* relationship set *contain*, *contain* should also be represented as a separate table
 - ▣ *contain*=(O-number, P-number, quantity of ordered))

Appendix 6-1 Example 1

- Therefore, the E-R diagram can be reduced into the following five tables
 - ▣ **customer**=(C-number, c-name, discount, balance)
 - ▣ **ship-to-addresses** = (C-number, ship-to-addresses)
 - ▣ **order**=(O-number, C-number, ship-to-address, year, month, day)
 - ▣ **product**=(P-number, p-name, plant, quantity-on-hand)
 - ▣ **contain**=(O-number, P-number, quantity of ordered))

Outline

